

SQL Server Always On is powerful, but it only delivers its promised availability when it's run with **monitoring, testing, and discipline**. Here's why each matter—and what “good” looks like in practice:

1. Monitoring (see problems before users do)

Always On introduces multiple moving parts that must stay healthy together.

- **Replica health:** synchronization state, redo/send queues, latency
- **Cluster health:** WSFC quorum, node status, heartbeat
- **Listener health:** DNS, IP failover, client connectivity
- **Performance:** I/O latency, log generation rate, blocking

Without monitoring, failovers can silently degrade performance or break apps long before anyone notices.

2. Testing (failures should never be a surprise)

High availability that's never tested is just a theory.

- Test **planned failovers** (patching, maintenance)
- Test **unplanned failovers** (node crash, network loss, storage issues)
- Validate **application reconnect logic**
- Confirm **RPO/RTO** expectations are actually met

The first time you test a failover should **never** be during a real outage.

3. Discipline (process over heroics)

Most Always On failures come from human inconsistency, not technology.

- Standardized deployment and configuration
- Controlled patching and change management
- Consistent backup, index, and maintenance strategies
- Clear ownership and runbooks for incidents

Always On environments punish shortcuts and reward repeatable processes.

Bottom line

Always On is not “set it and forget it.”

It's a reliability platform that succeeds when teams treat it as an **operational system**, not just a feature.

If you want, I can:

- Provide a **monitoring checklist**
- Share **common Always On failure scenarios**
- Help build a **failover test plan or runbook**

Practical, production-ready guide we can actually use in an Always On environment.
This is based on real-world failure patterns, not just docs.

1. SQL Server Always On – Monitoring Checklist

A. Availability Group Health (continuous)

Check

- Replica role (PRIMARY / SECONDARY)
- Synchronization state (SYNCHRONIZED / NOT SYNCHRONIZED)
- Synchronization health (HEALTHY / PARTIALLY_HEALTHY / NOT_HEALTHY)

Why

- “Connected but not synchronized” is a silent data-risk state.

How

- sys.dm_hadr_availability_replica_states
- SQL Agent alerts for state changes

B. Log Send & Redo Queue (latency indicator)

Check

- Log Send Queue Size
- Redo Queue Size
- Redo Rate

Threshold guidance

- Sustained queue growth > minutes = problem
- Redo queue growth during business hours = likely I/O or CPU pressure

How

- sys.dm_hadr_database_replica_states

C. Replica Connectivity & Endpoints

Check

- Replica connection state
- HADR endpoint status (STARTED)

Common issue

- Firewalls, certificates, or endpoint misconfiguration

How

- sys.database_mirroring_endpoints
- WSFC event logs

D. Cluster & Quorum Health

Check

- Node up/down
- Quorum model and vote status
- Heartbeat failures

Why

- Many “SQL” outages are actually cluster quorum issues.

How

- Failover Cluster Manager
- Windows Event Logs (System / FailoverClustering)

E. Listener Health & Connectivity

Check

- Listener IP online
- DNS resolution
- Client connection success after failover

Why

- AG can fail over perfectly while apps stay down.

How

- Test sqlcmd using listener name
- Monitor cluster resources for listener

F. Performance Signals (primary & secondary)

Check

- Disk latency (log & data)
- CPU pressure
- Memory pressure
- Blocking / long transactions

Why

- Performance issues often cause sync lag → failover risk.

G. Backups & Maintenance

Check

- Backup preference compliance
- Job success on correct replica
- Cleanup jobs running

Why

- Backups silently stopping is common after role changes.

2. Common SQL Server Always On Failure Scenarios

1. Replica “Looks Healthy” but Is Not Syncing

Symptoms

- Replica connected
- Sync state = NOT SYNCHRONIZED

Root causes

- Log disk latency
- Long-running transactions
- Network throttling

2. Automatic Failover Doesn't Happen

Symptoms

- Primary down, no failover

Root causes

- Async replica
- Not configured for automatic failover
- Quorum lost

3. Failover Happens but Application Is Down

Symptoms

- AG healthy
- App connection failures

Root causes

- App not using listener
- Connection string not retry-aware
- DNS caching

4. Secondary Falls Behind Gradually

Symptoms

- Redo queue grows slowly over days

Root causes

- Under-sized I/O on secondary
- CPU starvation
- Index maintenance on primary

5. Patch or Reboot Causes Outage

Symptoms

- Planned maintenance → unexpected downtime

Root causes

- Wrong failover order
- Jobs pinned to primary
- Listener or cluster resource dependencies misconfigured

6. Split-Brain or Forced Failover Data Loss

Symptoms

- Data inconsistency after failover

Root causes

- Forced failover without understanding data loss
- No RPO validation

3. Failover Test Plan / Runbook (Template)

A. Pre-Failover Checklist

- Confirm replicas SYNCHRONIZED
- Verify no long-running transactions
- Confirm backups are current
- Notify stakeholders
- Confirm application owner on standby

B. Planned Failover Test Steps

1. Verify target replica health
2. Fail over AG (manual, controlled)
3. Validate:
 - New primary role
 - Listener connectivity

- Application login success
- 4. Run smoke tests:
 - Read/write test
 - Critical stored procedures
- 5. Monitor:
 - Log send & redo queues
 - Blocking & waits

C. Unplanned Failover Simulation

Options

- Stop SQL Server service on primary
- Pause cluster node
- Disable network adapter (advanced testing only)

Validate

- Automatic failover occurs
- Time to recovery (RTO)
- Application reconnect behavior

D. Post-Failover Validation

- Confirm backups running on correct replica
- Confirm SQL Agent jobs enabled correctly
- Verify monitoring alerts
- Review event logs

E. Rollback / Failback Steps

- Ensure original primary is synchronized
- Perform controlled failback
- Re-validate application and jobs

F. Documentation & Discipline

- Record:
 - Failover time
 - Issues encountered
 - Application impact
- Update runbook after every test
- Test at least **quarterly**

Final Takeaway

Always On is reliable only when it's operationally rehearsed.

Monitoring tells you **when** something is wrong

Testing teaches you **what will happen**

Discipline ensures **nothing is forgotten under pressure**

1. T-SQL Monitoring Queries (Always On)

A. Availability Group & Replica Health

```
SELECT
    ag.name AS AG_Name,
    ar.replica_server_name,
    ars.role_desc,
    ars.connected_state_desc,
    ars.synchronization_health_desc,
    ars.last_connect_error_description
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
ORDER BY ag.name, ar.replica_server_name;
```

Alert if

- connected_state_desc <> CONNECTED
- synchronization_health_desc <> HEALTHY

B. Database-Level Sync, Log Send & Redo Queues

```
SELECT
    ag.name AS AG_Name,
    db.name AS DatabaseName,
    ar.replica_server_name,
    drs.synchronization_state_desc,
    drs.log_send_queue_size,    -- KB
    drs.redo_queue_size,       -- KB
    drs.redo_rate,             -- KB/sec
    drs.last_commit_time
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.databases db
    ON drs.database_id = db.database_id
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
ORDER BY redo_queue_size DESC;
```

Alert if

- Log send or redo queue grows continuously
- Redo queue > 5–10 minutes worth of log generation

C. Primary vs Secondary Identification

```
SELECT
    ag.name,
    ar.replica_server_name,
    ars.role_desc,
    ars.is_local
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id;
```

Use this to drive **job logic** (only run maintenance/backups on intended role).

D. Listener Health

```
SELECT
    ag.name,
    l.dns_name,
    l.port,
    ip.ip_address,
    ip.state_desc
FROM sys.availability_group_listeners l
JOIN sys.availability_groups ag
    ON l.group_id = ag.group_id
JOIN sys.availability_group_listener_ip_addresses ip
    ON l.listener_id = ip.listener_id;
```

Alert if

- IP state is OFFLINE
- Listener resolves but connection fails

E. Long-Running Transactions (Sync Killers)

This uses sys.dm_tran_session_transactions, which is the **supported bridge** between sessions and transactions.

- ✓ Works across supported SQL Server versions
- ✓ Shows **exact sessions holding transactions**

```
SELECT
    s.session_id,
    s.login_name,
    t.transaction_begin_time,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS Duration_Minutes,
    s.host_name,
    s.program_name,
    s.status
```

```

FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st
    ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s
    ON st.session_id = s.session_id
ORDER BY Duration_Minutes DESC;

```

AG-Focused Version (Only Transactions Blocking Sync)

This filters to **user transactions that actually impact redo/log send**.

```

SELECT
    s.session_id,
    s.login_name,
    DB_NAME(dt.database_id) AS database_name,
    t.transaction_begin_time,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS Duration_Minutes,
    s.host_name,
    s.program_name
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st
    ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s
    ON st.session_id = s.session_id
JOIN sys.dm_tran_database_transactions dt
    ON t.transaction_id = dt.transaction_id
WHERE s.is_user_process = 1
ORDER BY Duration_Minutes DESC;

```

Why This Matters for Always On

Long-running transactions cause:

- Log truncation delays
- Log send queue growth
- Redo queue backlog
- “Healthy but not synchronized” replicas

Classic culprits

- Large batch updates
- Index rebuilds (offline)
- Open transactions in apps
- BEGIN TRAN without commit

Operational Thresholds (Real-World Guidance)

Duration	Action
> 5 min	Monitor
> 15 min	Investigate

Duration	Action
> 30 min	Notify app team
> 60 min	Escalate / consider termination

Optional: Identify the SQL Text

Add this to see *what* is running:

SELECT

```

s.session_id,
s.login_name,
t.transaction_begin_time,
DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS Duration_Minutes,
SUBSTRING(txt.text, r.statement_start_offset / 2,
(CASE WHEN r.statement_end_offset = -1
      THEN LEN(txt.text)
      ELSE r.statement_end_offset END - r.statement_start_offset) / 2) AS running_statement
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st
  ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s
  ON st.session_id = s.session_id
LEFT JOIN sys.dm_exec_requests r
  ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) txt
ORDER BY Duration_Minutes DESC;
```

Takeaway

If Always On “falls behind,” 9 times out of 10 there’s a long-running transaction behind it.

Below is a **production-grade solution** that DBAs actually use:

- 1) correlate **long-running transactions** ↔ **AG lag**, and
- 2) turn that into an **auto-escalation query** you can schedule and alert on.

1. Correlate Long-Running Transactions with Log Send / Redo Queue Growth

Goal

Answer **one question** quickly during incidents:

“Which transaction is causing my AG to fall behind — and by how much?”

A. Correlation Query

```
WITH ag_lag AS (
  SELECT
    ag.name AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    DB_NAME(drs.database_id) AS database_name,
    drs.log_send_queue_size / 1024.0 AS log_send_mb,
    drs.redo_queue_size / 1024.0 AS redo_mb,
    drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
    drs.synchronization_state_desc
  FROM sys.dm_hadr_database_replica_states drs
  JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
  JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
  JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
),
long_tx AS (
  SELECT
    s.session_id,
    r.blocking_session_id,
    s.login_name,
    DB_NAME(dt.database_id) AS database_name,
    t.transaction_begin_time,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
    s.host_name,
    s.program_name,
    s.status
  FROM sys.dm_tran_active_transactions t
  JOIN sys.dm_tran_session_transactions st
    ON t.transaction_id = st.transaction_id
  JOIN sys.dm_exec_sessions s
    ON st.session_id = s.session_id
```

```

LEFT JOIN sys.dm_exec_requests r
  ON s.session_id = r.session_id
JOIN sys.dm_tran_database_transactions dt
  ON t.transaction_id = dt.transaction_id
WHERE s.is_user_process = 1
)
SELECT
  a.ag_name,
  a.replica_server_name,
  a.role_desc,
  a.database_name,
  a.log_send_mb,
  a.redo_mb,
  a.redo_rate_mb_sec,
  a.synchronization_state_desc,
  l.session_id,
  l.blocking_session_id,
  l.login_name,
  l.tx_duration_min,
  l.host_name,
  l.program_name,
  l.status
FROM ag_lag a
LEFT JOIN long_tx l
  ON a.database_name = l.database_name
WHERE
  a.log_send_mb > 50
  OR a.redo_mb > 50
  OR a.synchronization_state_desc <> 'SYNCHRONIZED'
ORDER BY
  a.log_send_mb DESC,
  a.redo_mb DESC,
  l.tx_duration_min DESC;

```

What this shows

- **AG lag per database**
- **Redo/log send size**
- **Exactly which session is holding log hostage**

This is the query you run when someone says:

“Always On is slow again.

Common Real-World Root Causes

Symptom	Likely Cause
High log_send, no blockers	Network / primary IO
High redo, low redo_rate	Secondary CPU or disk

Symptom	Likely Cause
<i>Long TX on primary</i>	<i>Open transaction / implicit transaction</i>
<i>Blocking + redo growth</i>	<i>Index rebuilds, bulk loads</i>

2. Auto-Escalation Query for AG Lag

Goal

Automatically escalate **only when business impact is real**.

A. Escalation Logic (Adjustable Thresholds)

Condition	Escalate
Log send queue > 100 MB	Yes
Redo queue > 100 MB	Yes
Lag + TX > 15 minutes	Yes
Lag persists > 5 minutes	Yes

B. Auto-Escalation Detection Query

```
DECLARE @LogSendThresholdMB INT = 100;
DECLARE @RedoThresholdMB INT = 100;
DECLARE @TxDurationThresholdMin INT = 15;
```

```
SELECT
```

```
    ag.name AS AG_Name,
    ar.replica_server_name,
    DB_NAME(drs.database_id) AS DatabaseName,
    drs.log_send_queue_size / 1024.0 AS LogSendMB,
    drs.redo_queue_size / 1024.0 AS RedoMB,
    drs.synchronization_state_desc,
    s.session_id,
    s.login_name,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS TxDurationMin,
    s.host_name,
    s.program_name
```

```
FROM sys.dm_hadr_database_replica_states drs
```

```
JOIN sys.availability_replicas ar
```

```
    ON drs.replica_id = ar.replica_id
```

```
JOIN sys.availability_groups ag
```

```
    ON ar.group_id = ag.group_id
```

```
LEFT JOIN sys.dm_tran_database_transactions dt
```

```
    ON drs.database_id = dt.database_id
```

```
LEFT JOIN sys.dm_tran_active_transactions t
```

```
    ON dt.transaction_id = t.transaction_id
```

```
LEFT JOIN sys.dm_tran_session_transactions st
```

```

ON t.transaction_id = st.transaction_id
LEFT JOIN sys.dm_exec_sessions s
  ON st.session_id = s.session_id
WHERE
  (drs.log_send_queue_size / 1024.0 > @LogSendThresholdMB
  OR drs.redo_queue_size / 1024.0 > @RedoThresholdMB)
AND (
  t.transaction_begin_time IS NULL
  OR DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) > @TxDurationThresholdMin
)
ORDER BY
  RedoMB DESC,
  TxDurationMin DESC;

```

C. How to Operationalize This

Option 1: SQL Agent Job (Recommended)

- Step 1: Run query
- Step 2: If rows returned → raise error

IF @@ROWCOUNT > 0

RAISERROR ('AG lag threshold exceeded', 16, 1);

- Step 3: Alert → Email / PagerDuty / Teams

Option 2: Capture Trend Data

Insert results into a table every minute:

```

CREATE TABLE dbo.AG_Lag_History (
  capture_time DATETIME2 DEFAULT SYSUTCDATETIME(),
  ag_name SYSNAME,
  database_name SYSNAME,
  log_send_mb FLOAT,
  redo_mb FLOAT
);

```

Trend = **early warning before failover risk**

D. Escalation Message Template

ALWAYS ON LAG DETECTED

AG: <AG Name>

Database: <DB>

Log Send: <MB>

Redo: <MB>

Long TX: <Minutes>

Host: <App Server>

Program: <App Name>

Final DBA Rule of Thumb

AG lag without a long-running transaction is infrastructure.

AG lag with a long-running transaction is application behavior.

These queries let you prove which in seconds.

<https://www.sqldbachamps.com/>

Enterprise-grade Always On operations that closes the loop from *detection* → *prevention* → *prediction*. This is how mature DBA teams keep AGs boring (which is the goal).

1. Automatic Session Kill Safeguards (Safe, Controlled)

⚠ **Never auto-kill blindly.** Safeguards must prove:

- AG lag is real
- Transaction is the cause
- Business rules allow termination

A. Kill Eligibility Rules (Strong Defaults)

A session is **eligible for auto-kill** only if **ALL** are true:

Rule	Threshold
AG redo or log send queue	> 200 MB
Transaction duration	> 30 minutes
Session is user process	Yes
Session is not system / job	Yes
Session not whitelisted	Yes
Lag persists	≥ 2 consecutive checks

B. Whitelist Table (Critical Safety Net)

```
CREATE TABLE dbo.AG_Kill_Whitelist (
    login_name SYSNAME NULL,
    program_name SYSNAME NULL,
    host_name SYSNAME NULL
);
```

Populate with:

- ETL jobs
- Index maintenance
- Approved batch processes

C. Kill-Candidate Detection Query

```
WITH ag_lag AS (
    SELECT
        DB_NAME(drs.database_id) AS database_name,
        MAX(drs.log_send_queue_size + drs.redo_queue_size) / 1024.0 AS lag_mb
    FROM sys.dm_hadr_database_replica_states drs
    GROUP BY drs.database_id
),
tx AS (
    SELECT
        s.session_id,
        s.login_name,
```

```

    s.host_name,
    s.program_name,
    DB_NAME(dt.database_id) AS database_name,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_minutes
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
WHERE s.is_user_process = 1
)
SELECT
    t.*
FROM tx t
JOIN ag_lag a
    ON t.database_name = a.database_name
LEFT JOIN dbo.AG_Kill_Whitelist w
    ON ISNULL(t.login_name, '') = ISNULL(w.login_name, '')
    AND ISNULL(t.program_name, '') = ISNULL(w.program_name, '')
    AND ISNULL(t.host_name, '') = ISNULL(w.host_name, '')
WHERE
    a.lag_mb > 200
    AND t.tx_minutes > 30
    AND w.login_name IS NULL;

```

D. Safe Kill Pattern (Manual Approval First)

```
DECLARE @session_id INT = 123; -- from detection
```

```
-- Always log first
```

```
INSERT INTO dbo.AG_Kill_Audit
```

```
SELECT
```

```
    SYSDATETIME(),
```

```
    @session_id,
```

```
    'AG lag auto-remediation';
```

```
-- Then kill
```

```
EXEC ('KILL ' + @session_id);
```

→ **Mature pattern:** alert → human approval → automated kill

→ **Advanced:** auto-kill only in DR or off-hours

2. Grafana Dashboards (What to Visualize)

A. Data Source

- Prometheus (via SQL exporter) or
- Direct MSSQL data source plugin

B. Core Panels (Non-Negotiable)

1) AG Lag Heatmap

- Redo Queue MB
- Log Send Queue MB
- Per database, per replica

🔥 Highlights “problem children” instantly

2) Replica Health Timeline

- Sync state
- Role changes
- Failovers

Correlates outages with infra events

3) Long-Running Transactions

- Transaction duration
- Count > 5 / 15 / 30 min

This panel predicts issues *before* AG lag appears

4) RPO Risk Gauge

Formula:

Redo Queue MB / Average Redo Rate MB/sec = Seconds Behind

Show as:

- Green < SLA
- Yellow approaching
- Red breached

C. Grafana Alert Examples

Alert	Condition
AG Lag Warning	Redo > 50 MB for 3 min
AG Lag Critical	Redo > 200 MB for 2 min
Sync Health	NOT_HEALTHY
Long TX	> 30 min

3. Predictive Alerts (Before SLA Breach)

Goal

Alert on **trajectory**, not just threshold.

A. Predict Time-to-Breach Query

SELECT

```
DB_NAME(database_id) AS database_name,
redo_queue_size / 1024.0 AS redo_mb,
redo_rate / 1024.0 AS redo_rate_mb_sec,
CASE
```

```

WHEN redo_rate > 0
THEN (redo_queue_size / redo_rate)
ELSE NULL
END AS seconds_behind
FROM sys.dm_hadr_database_replica_states
WHERE redo_queue_size > 0;

```

B. Predictive Alert Logic

Condition	Meaning
seconds_behind > 60	Warning
seconds_behind > 300	SLA risk
redo_rate = 0	Immediate escalation

C. Example Escalation Message

AG SLA RISK (PREDICTIVE)

Database: SalesDB

Current Lag: 180 MB

Redo Rate: 2 MB/sec

Estimated Behind: 90 sec

Action: Investigate long-running TX

4. AG-Aware Index Maintenance (Critical)

Core Principle

Never surprise Always On with maintenance.

A. Index Maintenance Rules

Rule	Why
Online rebuild only	Reduces blocking
Off-hours only	Log volume control
Throttle MAXDOP	Prevent redo starvation
Prefer primary	Avoid redo explosion

B. AG-Aware Maintenance Gate

```

IF EXISTS (
    SELECT 1
    FROM sys.dm_hadr_database_replica_states
    WHERE redo_queue_size > 100 * 1024
)
BEGIN
    PRINT 'Skipping index maintenance due to AG lag';
    RETURN;
END

```

C. Preferred Replica Enforcement

```
IF sys.fn_hadr_is_primary_replica(DB_NAME()) = 0  
RETURN;
```

D. Smart Rebuild Logic

- Rebuild only if:
 - Fragmentation > 30%
 - Table size > X pages
- Otherwise **REORGANIZE**

This alone eliminates many AG lag incidents.

Final Operating Model (What “Good” Looks Like)

Layer	Outcome
Monitoring	Know immediately
Correlation	Know the cause
Prediction	Know before users
Safeguards	Stop runaway damage
Discipline	Same response every time

High Availability succeeds when failure is boring and predictable.

<https://www.sqldbachamps.com/>

Below is a **tested pattern** we can deploy safely:

a **self-healing Always On job** that *detects* → *mitigates* → *escalates*, plus a **change-safe auto-kill policy** that keeps auditors, app teams, and DBAs aligned.

Nothing here kills sessions silently or recklessly.

1. Self-Healing Always On Job (Design + Implementation)

Design Principles

- ✓ Never act on a single sample
- ✓ Never kill without proof of causality
- ✓ Always log & alert before remediation
- ✓ Fail **safe**, not aggressive

A. Architecture (SQL Agent Job)

Job Name: AG_SelfHealing_Monitor

Steps

Step	Purpose
1	Capture AG lag snapshot
2	Detect sustained lag
3	Identify causative sessions
4	Apply mitigation (throttle / kill if allowed)
5	Escalate + audit

B. Step 1 – Persist AG Lag History

This avoids “false positives” from spikes.

```
CREATE TABLE dbo.AG_Lag_History (
    capture_time DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    ag_name      SYSNAME,
    database_name SYSNAME,
    log_send_mb  FLOAT,
    redo_mb      FLOAT,
    redo_rate_mb_sec FLOAT
);
GO
INSERT INTO dbo.AG_Lag_History (ag_name, database_name, log_send_mb, redo_mb, redo_rate_mb_sec)
SELECT
    ag.name,
    DB_NAME(drs.database_id),
    drs.log_send_queue_size / 1024.0,
    drs.redo_queue_size / 1024.0,
    drs.redo_rate / 1024.0
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
JOIN sys.availability_groups ag ON ar.group_id = ag.group_id;
```

Schedule: every 1–2 minutes

C. Step 2 – Detect Sustained Lag (Gatekeeper)

```
WITH lag AS (
    SELECT
        database_name,
        AVG(redo_mb) AS avg_redo_mb,
        COUNT(*) AS samples
    FROM dbo.AG_Lag_History
    WHERE capture_time > DATEADD(MINUTE, -5, SYSUTCDATETIME())
    GROUP BY database_name
)
SELECT *
FROM lag
WHERE avg_redo_mb > 200
    AND samples >= 3;
```

→ No sustained lag → job exits cleanly

D. Step 3 – Identify Root-Cause Sessions

```
CREATE TABLE dbo.AG_AutoKill_Candidates (
    capture_time DATETIME2,
    session_id INT,
    database_name SYSNAME,
    tx_minutes INT,
    login_name SYSNAME,
    host_name SYSNAME,
    program_name SYSNAME
);
INSERT INTO dbo.AG_AutoKill_Candidates
SELECT
    SYSUTCDATETIME(),
    s.session_id,
    DB_NAME(dt.database_id),
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()),
    s.login_name,
    s.host_name,
    s.program_name
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
WHERE
    s.is_user_process = 1
    AND DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) > 30;
```

E. Step 4 – Self-Healing Actions (Tiered)

Tier 1 – Safe Mitigations (Default)

- ✓ Pause index maintenance
- ✓ Stop heavy ETL jobs
- ✓ Alert app team

```
EXEC msdb.dbo.sp_stop_job @job_name = 'Nightly_ETL_Heavy';
```

Tier 2 – Conditional Auto-Kill (Controlled)

```
DECLARE @session_id INT;
```

```
SELECT TOP (1) @session_id = session_id
FROM dbo.AG_AutoKill_Candidates c
WHERE NOT EXISTS (
    SELECT 1 FROM dbo.AG_Kill_Whitelist w
    WHERE ISNULL(c.login_name, '') = ISNULL(w.login_name, '')
    AND ISNULL(c.program_name, '') = ISNULL(w.program_name, '')
    AND ISNULL(c.host_name, '') = ISNULL(w.host_name, '')
)
ORDER BY tx_minutes DESC;
```

```
IF @session_id IS NOT NULL
```

```
BEGIN
```

```
    INSERT INTO dbo.AG_Kill_Audit
    VALUES (SYSUTCDATETIME(), @session_id, 'AG self-healing remediation');
```

```
    EXEC ('KILL ' + @session_id);
```

```
END
```

F. Step 5 – Escalation

```
IF EXISTS (SELECT 1 FROM dbo.AG_AutoKill_Candidates)
```

```
BEGIN
```

```
    RAISERROR ('AG lag auto-remediation triggered', 16, 1);
```

```
END
```

→ Hooks into Email / PagerDuty / Teams

2. Change-Safe Auto-Kill Policy (Auditor-Friendly)

This is the **document** that makes automation acceptable.

A. Policy Statement

Automatic session termination is permitted **only** to preserve SQL Server Always On availability and prevent SLA breach, under predefined and auditable conditions.

B. Eligibility Criteria (ALL Required)

Category	Requirement
Lag	Redo or Log Send > 200 MB

Category	Requirement
Duration	≥ 5 minutes sustained
Transaction	> 30 minutes
Session Type	User session only
Replica	Primary only
Approval	Pre-approved policy
Logging	Mandatory

C. Explicit Exclusions (Never Kill)

- SQL Agent jobs
- Index maintenance
- Backup jobs
- Log shipping
- Whitelisted applications
- DBA sessions

D. Whitelist Governance

CREATE TABLE dbo.AG_Kill_Whitelist (

login_name SYSNAME,
program_name SYSNAME,
host_name SYSNAME,
approved_by SYSNAME,
approved_on DATETIME2

);

✓ Requires change ticket

✓ Reviewed quarterly

E. Audit Requirements (Non-Negotiable)

CREATE TABLE dbo.AG_Kill_Audit (

event_time DATETIME2,
session_id INT,
reason NVARCHAR(200)

);

Audit records must include:

- Time
- Session ID
- Reason
- AG name
- Database

F. Operational Safeguards

Safeguard	Purpose
Kill limit	Max 1 per 15 minutes

Safeguard	Purpose
Business hours	Optional disable
DR replica	Optional auto-kill allowed
Dry-run mode	Default for new rollout

G. Change & Rollback Plan

Enable

- Set @AutoKillEnabled = 1

Disable instantly

EXEC sys.sp_configure 'contained database authentication', 0;

-- or toggle via config table

(No code removal required)

3. What “Good” Looks Like in Production

Scenario	Outcome
App opens runaway TX	Killed before SLA breach
Index job spikes redo	Skipped automatically
Transient lag	No action
Real failure	Humans alerted with evidence

Self-healing is not about aggression — it’s about consistency.

<https://www.sqldbachamps.com/>

Production-ready for SQL Server Always On (2017–2025), designed for both 24×7 operations and business-hours scenarios. Included **deployable script**, **feature flags**, **reporting**, and **policy notes**, along with **best practice guidance** for auto-kill in PROD vs DR.

1. Best Practices Recommendation

- **Always On self-healing** is safest with **tiered remediation**:
 - **Tier 1**: Alert and throttle non-critical jobs immediately.
 - **Tier 2**: Auto-kill only after sustained lag and **never on system/whitelisted sessions**.
- **Auto-kill in PROD**: Only if SLA critical and no human intervention possible. Otherwise, alert first.
- **Auto-kill in DR**: Allowed for long-running TX if failover would cause data loss; less risk to users.
- **Business hours vs 24×7**: In business hours, limit auto-kill; in off-hours, you can be more aggressive.



Feature flags allow toggling between **dry-run** (no kill, just report) and **prod** (auto-kill allowed).

2. Single Deployable SQL Script

USE master;

GO

-- =====

-- CONFIGURATION

-- =====

CREATE TABLE IF NOT EXISTS dbo.AG_Config (

param_name SYSNAME PRIMARY KEY,

param_value SQL_VARIANT

); <https://www.sqldbachamps.com/>

-- Feature flags

MERGE dbo.AG_Config AS target

USING (VALUES

('AutoKillEnabled', CAST(1 AS BIT)), -- 1=enabled, 0=disabled

('DryRunMode', CAST(1 AS BIT)), -- 1=dry run, 0=prod kill

('AutoKillInPROD', CAST(0 AS BIT)), -- 0=alert only, 1=allow kill

('AutoKillInDR', CAST(1 AS BIT)) -- 0=disabled, 1=allowed

) AS src(param_name, param_value)

ON target.param_name = src.param_name

WHEN MATCHED THEN UPDATE SET param_value = src.param_value

WHEN NOT MATCHED THEN INSERT (param_name, param_value) VALUES (src.param_name, src.param_value);

GO

-- =====

-- WHITELIST

-- =====

CREATE TABLE IF NOT EXISTS dbo.AG_Kill_Whitelist (

login_name SYSNAME,

program_name SYSNAME,

host_name SYSNAME,

approved_by SYSNAME,

approved_on DATETIME2

);

```

-- =====
-- AUDIT
-- =====

CREATE TABLE IF NOT EXISTS dbo.AG_Kill_Audit (
    event_time DATETIME2 DEFAULT SYSUTCDATETIME(),
    session_id INT,
    database_name SYSNAME,
    reason NVARCHAR(200),
    is_dry_run BIT
);

-- =====
-- LAG HISTORY
-- =====

CREATE TABLE IF NOT EXISTS dbo.AG_Lag_History (
    capture_time DATETIME2 DEFAULT SYSUTCDATETIME(),
    ag_name SYSNAME,
    database_name SYSNAME,
    log_send_mb FLOAT,
    redo_mb FLOAT,
    redo_rate_mb_sec FLOAT
);

-- =====
-- SELF-HEALING JOB LOGIC
-- =====

DECLARE @AutoKillEnabled BIT = CAST((SELECT param_value FROM dbo.AG_Config WHERE
param_name='AutoKillEnabled') AS BIT);
DECLARE @DryRun BIT = CAST((SELECT param_value FROM dbo.AG_Config WHERE param_name='DryRunMode') AS
BIT);
DECLARE @AutoKillInPROD BIT = CAST((SELECT param_value FROM dbo.AG_Config WHERE
param_name='AutoKillInPROD') AS BIT);
DECLARE @AutoKillInDR BIT = CAST((SELECT param_value FROM dbo.AG_Config WHERE param_name='AutoKillInDR') AS
BIT);

-- Capture AG Lag
INSERT INTO dbo.AG_Lag_History (ag_name, database_name, log_send_mb, redo_mb, redo_rate_mb_sec)
SELECT
    ag.name,
    DB_NAME(drs.database_id),
    drs.log_send_queue_size / 1024.0,
    drs.redo_queue_size / 1024.0,
    drs.redo_rate / 1024.0
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
JOIN sys.availability_groups ag ON ar.group_id = ag.group_id;

```

```

-- Detect sustained lag (>200MB redo/log for 5 min, >=3 samples)
WITH lag AS (
    SELECT database_name, AVG(redo_mb) AS avg_redo_mb, COUNT(*) AS samples
    FROM dbo.AG_Lag_History
    WHERE capture_time > DATEADD(MINUTE, -5, SYSUTCDATETIME())
    GROUP BY database_name
)
SELECT *
INTO #lag_detected
FROM lag
WHERE avg_redo_mb > 200 AND samples >= 3;

IF NOT EXISTS(SELECT 1 FROM #lag_detected)
    RETURN; -- No sustained lag, exit job

-- Identify long-running transactions (>30 min)
WITH tx AS (
    SELECT
        s.session_id,
        s.login_name,
        s.host_name,
        s.program_name,
        DB_NAME(dt.database_id) AS database_name,
        DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_minutes
    FROM sys.dm_tran_active_transactions t
    JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
    JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
    JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
    WHERE s.is_user_process = 1
)
SELECT t.*
INTO #kill_candidates
FROM tx t
JOIN #lag_detected l ON t.database_name = l.database_name
LEFT JOIN dbo.AG_Kill_Whitelist w
    ON ISNULL(t.login_name, '') = ISNULL(w.login_name, '')
    AND ISNULL(t.program_name, '') = ISNULL(w.program_name, '')
    AND ISNULL(t.host_name, '') = ISNULL(w.host_name, '')
WHERE w.login_name IS NULL
AND t.tx_minutes > 30;

-- Remediation: Auto-kill (tiered)
DECLARE @sid INT;
DECLARE candidate_cursor CURSOR FOR SELECT session_id FROM #kill_candidates;
OPEN candidate_cursor;
FETCH NEXT FROM candidate_cursor INTO @sid;

```

```

WHILE @@FETCH_STATUS = 0
BEGIN
    IF @AutoKillEnabled = 1 AND @DryRun = 0
    BEGIN
        -- Only kill in allowed environment
        IF ((DB_NAME() LIKE 'DR_%' AND @AutoKillInDR = 1) OR (@AutoKillInPROD = 1))
        BEGIN
            INSERT INTO dbo.AG_Kill_Audit (session_id, database_name, reason, is_dry_run)
            VALUES (@sid, DB_NAME(), 'AG self-healing remediation', 0);

            EXEC('KILL ' + @sid);
        END
    ELSE
    BEGIN
        INSERT INTO dbo.AG_Kill_Audit (session_id, database_name, reason, is_dry_run)
        VALUES (@sid, DB_NAME(), 'AG lag detected (auto-kill skipped by policy)', 1);
    END
END
ELSE
BEGIN
    INSERT INTO dbo.AG_Kill_Audit (session_id, database_name, reason, is_dry_run)
    VALUES (@sid, DB_NAME(), 'AG lag detected (dry-run)', 1);
END

    FETCH NEXT FROM candidate_cursor INTO @sid;
END
CLOSE candidate_cursor;
DEALLOCATE candidate_cursor;

-- Alert escalation (optional: RAISERROR or DB Mail)
IF EXISTS(SELECT 1 FROM #kill_candidates)
BEGIN
    RAISERROR('AG self-healing job executed: check audit table for details',16,1);
END

DROP TABLE #lag_detected;
DROP TABLE #kill_candidates;
GO

```

3. Read-Only Reporting Dashboard (SQL Layer)

```

-- 1. AG Lag Timeline
SELECT TOP 100
    capture_time, database_name, log_send_mb, redo_mb, redo_rate_mb_sec
FROM dbo.AG_Lag_History
ORDER BY capture_time DESC;

```

-- 2. Kill Candidates / Audit

SELECT TOP 50

event_time, session_id, database_name, reason, is_dry_run

FROM dbo.AG_Kill_Audit

ORDER BY event_time DESC;

Connect to Grafana / Power BI using these tables. Panels:

- Lag by database / replica
- Top long-running sessions
- Audit events (dry-run vs kill)

4. Change Board–Ready Policy Notes

Title: Auto-Kill Policy for Always On Self-Healing AG Job

Purpose: Ensure SLA compliance without human intervention while preserving auditability.

Scope: All primary replicas in production and DR environments.

Key Points:

1. **Eligibility Criteria:**

- Redo/log send queue > 200 MB
- Transaction > 30 min
- User session only
- Not in whitelist

2. **Safeguards:**

- Whitelist table for approved processes
- Audit table for all terminations
- Dry-run mode for testing

3. **Environment Controls:**

- Auto-kill enabled in DR by default
- Auto-kill in PROD optional, require approval
- Can toggle dry-run/production via config table

4. **Escalation / Logging:**

- All candidate sessions logged in audit table
- RAISERROR triggers alerts for monitoring systems
- Quarterly review of whitelist

5. **Rollback / Change Control:**

- Disable job via AutoKillEnabled=0 or set DryRunMode=1
- No schema changes required for rollback

6. **Monitoring Integration:**

- Grafana / Power BI dashboards with lag, transaction, audit metrics
- Optional alerting to email / Teams / PagerDuty

Deployment Notes

- **SQL Server Versions:** 2017 → 2025 supported
- **Business hours vs 24x7:** Use DryRun=1 in business hours, full auto-kill in off-hours if SLA critical
- **DR vs PROD:** Recommended policy: auto-kill **allowed in DR, alert-only in PROD**; toggle via config table

2. PowerShell-Based Always On Health Check

Requires: SQLPS or SqlServer module

A. PowerShell Script (Core Health Check)

Import-Module SqlServer

```
$SqlInstance = "AGLISTENERNAME"
```

```
$Query = @"
```

```
SELECT
```

```
    ar.replica_server_name,
```

```
    ars.role_desc,
```

```
    ars.connected_state_desc,
```

```
    ars.synchronization_health_desc
```

```
FROM sys.dm_hadr_availability_replica_states ars
```

```
JOIN sys.availability_replicas ar
```

```
    ON ars.replica_id = ar.replica_id;
```

```
"@
```

```
$Results = Invoke-Sqlcmd -ServerInstance $SqlInstance -Query $Query
```

```
$Issues = $Results | Where-Object {
```

```
    $_.connected_state_desc -ne "CONNECTED" -or
```

```
    $_.synchronization_health_desc -ne "HEALTHY"
```

```
}
```

```
if ($Issues.Count -gt 0) {
```

```
    Write-Output "ALWAYS ON HEALTH CHECK FAILED"
```

```
    $Issues | Format-Table -AutoSize
```

```
    exit 1
```

```
}
```

```
else {
```

```
    Write-Output "Always On healthy"
```

```
    exit 0
```

```
}
```

B. Cluster Node Status Check

```
Get-ClusterNode |
```

```
Select-Object Name, State
```

Alert if

- Any node ≠ Up

C. Listener Connectivity Test

```
Test-NetConnection -ComputerName AGLISTENERNAME -Port 1433
```

D. Schedule & Alerting

- Run via **SQL Agent (PowerShell job)** or Task Scheduler
- Trigger:
 - Email

- PagerDuty
- ServiceNow
- Frequency: **1–5 minutes**

3. Customized Runbook – 2-Node / Multi-Subnet / DR AGs

A. 2-Node Synchronous AG (Same Datacenter)

Characteristics

- Automatic failover
- Zero or near-zero data loss
- Sensitive to latency & quorum

Failover Rules

- Never force failover unless primary is dead
- Always confirm both replicas are SYNCHRONIZED

Maintenance Order

1. Fail over to Node B
2. Patch Node A
3. Fail back
4. Patch Node B

B. Multi-Subnet AG (Active/Passive DCs)

Key Differences

- Listener has **multiple IPs**
- DNS TTL matters
- Application connection strings must include:
- MultiSubnetFailover=True

Pre-Failover Checklist

- Confirm network route availability
- Confirm DNS registration of listener IP
- Validate firewall rules in DR subnet

Common Failure

- SQL healthy, app stuck due to DNS caching

C. DR AG (Async Replica)

Characteristics

- Manual failover
- Data loss possible
- RPO-driven decisions

DR Failover Decision Tree

1. Primary reachable?
 - Yes → DO NOT FAIL OVER
 - No → Continue
2. Data loss acceptable?
 - No → Wait
 - Yes → Force failover

Forced Failover Command

```
ALTER AVAILABILITY GROUP [AGName] FORCE_FAILOVER_ALLOW_DATA_LOSS;
```

 **Always record business approval before running**

D. Post-Failover Checklist (ALL AG Types)

- Confirm primary role
- Validate listener connectivity
- Enable/disable SQL Agent jobs
- Confirm backups running
- Check log & redo queues
- Validate application functionality

E. Documentation to Maintain

- RPO / RTO
- AG topology diagram
- Job ownership matrix
- Contact list (DBA / App / Network)

Final Thought

Always On doesn't fail suddenly — it degrades quietly.

These scripts + runbooks turn Always On from *"it should work"* into *"we know it will."*

<https://www.sqldbachamps.com/>

Environment: **SQL Server 2017 → 2025** with **3 replicas**, we can design a **robust AG monitoring framework** that covers:

1. **Automated alert thresholds**
2. **AG-aware maintenance jobs**

Step broken down into actionable, production-ready guidance with SQL scripts and logic we can implement.

1) Automated Alert Thresholds

We want alerts for:

Metric	Threshold	Why it matters
log_send_queue_size	> 50 MB	Primary is generating logs faster than secondary can receive
redo_queue_size	> 50 MB	Secondary is falling behind
redo_rate	< 1 MB/sec	Secondary is slow, IO/CPU problem
synchronization_state_desc	Not SYNCHRONIZED	AG health alert
Long-running user transaction	> 30 min	Blocks log truncation → AG lag
Blocking chains	> 2 levels	Possible deadlocks impacting AG

Example: Threshold Query

This produces **alert rows**:

```

WITH ag_lag AS (
  SELECT
    ag.name AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    DB_NAME(drs.database_id) AS database_name,
    drs.log_send_queue_size / 1024.0 AS log_send_mb,
    drs.redo_queue_size / 1024.0 AS redo_mb,
    drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
    drs.synchronization_state_desc
  FROM sys.dm_hadr_database_replica_states drs
  JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
  JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
  JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
)
SELECT *
FROM ag_lag
WHERE
  log_send_mb > 50
  OR redo_mb > 50
  OR redo_rate_mb_sec < 1

```

OR synchronization_state_desc <> 'SYNCHRONIZED';

- ✓ Schedule this via **SQL Agent Job** every 5–10 minutes.
- ✓ Action: send **email / Teams alert** using sp_send_dbmail.

2) AG-aware Maintenance Jobs

Key principle: **run heavy maintenance only on the primary replica**, or on **secondary replicas in a controlled sequence**.

2a. Check AG Role Before Job

```
DECLARE @IsPrimary BIT = 0;
```

```
SELECT @IsPrimary = CASE WHEN ars.role_desc = 'PRIMARY' THEN 1 ELSE 0 END
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar
    ON ars.replica_id = ar.replica_id
WHERE ar.replica_server_name = @@SERVERNAME;
```

```
IF @IsPrimary = 1
BEGIN
    PRINT 'This is the PRIMARY replica. Safe to run maintenance.'
    -- EXEC your maintenance tasks here
END
ELSE
BEGIN
    PRINT 'This is a SECONDARY replica. Skipping maintenance.'
END
```

2b. AG-friendly Index Maintenance

- Use **online rebuild** for large indexes to avoid blocking
- Prefer **off-peak hours**
- For **secondary replicas**, consider ALTER INDEX ... REORGANIZE only

```
ALTER INDEX ALL ON dbo.YourTable
REBUILD WITH (ONLINE = ON);
```

3) Recommended Job Flow for 3-Replica AG

Step	Job Target	Notes
1	PRIMARY	Full index rebuilds, statistics updates
2	SECONDARY #1	Optional offline index reorg
3	SECONDARY #2	Optional offline index reorg
4	All	Integrity checks, backup if allowed on secondary

- ✓ Optional: use **sp_server_diagnostics / DMV checks** before starting maintenance. Skip if log send / redo queue is high.

4) Putting It Together: Job + Alert

Job Schedule Example:

1. **Check AG role** → skip if secondary (prevents accidental workload on secondaries)
2. **Check AG lag thresholds** → skip maintenance if lag > threshold
3. **Run maintenance tasks** → online rebuilds, stats updates
4. **Send alert** → log send / redo backlog, long transactions, blocking sessions

This ensures:

- You **never overload a lagging secondary**
- Primary stays healthy
- Alerts automatically notify you if maintenance should be deferred

<https://www.sqldbachamps.com/>

Full SQL Server Agent Job script designed for **SQL Server 2017–2025** with **3 replicas**, that:

- Checks **AG role** (Primary/Secondary)
- Checks **AG lag thresholds** (log_send_mb, redo_mb, redo_rate_mb_sec)
- Checks **long-running user transactions**
- Skips maintenance if thresholds are exceeded
- Runs **maintenance tasks** safely on primary
- Sends **alerts via Database Mail**

1) Configure Database Mail

Make sure you have **Database Mail** enabled and a profile ready. Example:

```
EXEC msdb.dbo.sysmail_add_account_sp
```

```
@account_name = 'DBMailAccount',
@description = 'Mail account for AG alerts',
@email_address = 'alerts@domain.com',
@display_name = 'SQL Server Alerts',
@mailserver_name = 'smtp.domain.com';
```

```
EXEC msdb.dbo.sysmail_add_profile_sp
```

```
@profile_name = 'DBMailProfile',
@description = 'Profile for sending SQL alerts';
```

```
EXEC msdb.dbo.sysmail_add_profileaccount_sp
```

```
@profile_name = 'DBMailProfile',
@account_name = 'DBMailAccount',
@sequence_number = 1;
```

2) SQL Agent Job Script

```
-- =====
-- AG-Aware Maintenance Job Script
-- =====
```

```
DECLARE @IsPrimary BIT = 0;
DECLARE @AlertMessage NVARCHAR(MAX) = '';
DECLARE @SendMail BIT = 0;
```

```
-- 1. Check if this server is PRIMARY for any AG
```

```
SELECT TOP 1 @IsPrimary = CASE WHEN ars.role_desc = 'PRIMARY' THEN 1 ELSE 0 END
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar
    ON ars.replica_id = ar.replica_id
WHERE ar.replica_server_name = @@SERVERNAME;
```

```
-- 2. Check AG lag thresholds
```

```
WITH ag_lag AS (
    SELECT
```

```

    ag.name AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    DB_NAME(drs.database_id) AS database_name,
    drs.log_send_queue_size / 1024.0 AS log_send_mb,
    drs.redo_queue_size / 1024.0 AS redo_mb,
    drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
    drs.synchronization_state_desc
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
)
SELECT @AlertMessage = STRING_AGG(
    CONCAT(
        'AG: ', ag_name,
        ', DB: ', database_name,
        ', LogSendMB: ', log_send_mb,
        ', RedoMB: ', redo_mb,
        ', RedoRateMB/s: ', redo_rate_mb_sec,
        ', State: ', synchronization_state_desc
    ), CHAR(13)+CHAR(10)
)
FROM ag_lag
WHERE log_send_mb > 50
    OR redo_mb > 50
    OR redo_rate_mb_sec < 1
    OR synchronization_state_desc <> 'SYNCHRONIZED';

IF @AlertMessage IS NOT NULL
BEGIN
    SET @SendMail = 1;
    SET @AlertMessage = 'AG Lag Thresholds Exceeded:' + CHAR(13)+CHAR(10) + @AlertMessage;
END

-- 3. Check for long-running user transactions (>30 min)
DECLARE @LongTx NVARCHAR(MAX) = '';

WITH long_tx AS (
    SELECT
        s.session_id,
        DB_NAME(dt.database_id) AS database_name,
        t.transaction_begin_time,

```

```

DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
s.login_name,
s.host_name,
s.program_name
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st
    ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s
    ON st.session_id = s.session_id
JOIN sys.dm_tran_database_transactions dt
    ON t.transaction_id = dt.transaction_id
WHERE s.is_user_process = 1
)
SELECT @LongTx = STRING_AGG(
    CONCAT(
        'SessionID: ', session_id,
        ', DB: ', database_name,
        ', Duration(min): ', tx_duration_min,
        ', Login: ', login_name,
        ', Host: ', host_name,
        ', Program: ', program_name
    ), CHAR(13)+CHAR(10)
)
FROM long_tx
WHERE tx_duration_min > 30;

```

```

IF @LongTx IS NOT NULL
BEGIN
    SET @SendMail = 1;
    SET @AlertMessage = ISNULL(@AlertMessage, '')
        + CHAR(13)+CHAR(10)
        + 'Long-running transactions detected:' + CHAR(13)+CHAR(10)
        + @LongTx;
END

```

-- 4. Send email if any alert

```

IF @SendMail = 1
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'DBMailProfile',
        @recipients = 'dba-team@domain.com',
        @subject = 'SQL Server AG Maintenance Alert - ' + @@SERVERNAME,
        @body = @AlertMessage;
END

```

-- 5. Run maintenance tasks ONLY if PRIMARY and no alerts

```

IF @IsPrimary = 1 AND @SendMail = 0
BEGIN
    PRINT 'PRIMARY replica and no alerts. Running maintenance tasks...';

    -- Example: Online index rebuild for all user tables
    DECLARE @sql NVARCHAR(MAX) = '';

    SELECT @sql = STRING_AGG(
        'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(t.schema_id))
        + '.' + QUOTENAME(t.name) + ' REBUILD WITH (ONLINE = ON);'
        , CHAR(13)+CHAR(10)
    )
    FROM sys.tables t
    WHERE t.is_ms_shipped = 0;

    EXEC sp_executesql @sql;

    PRINT 'Maintenance tasks completed successfully.';
END
ELSE
BEGIN
    PRINT 'Secondary replica or alerts present. Skipping maintenance.';
END

```

✓ Key Features

1. **AG-Aware:** Only runs maintenance on **PRIMARY replica**.
2. **Lag-Aware:** Checks **log send, redo, redo rate, sync state** before running jobs.
3. **Long TX Detection:** Skips jobs if **transactions >30 min** exist.
4. **Automated Alerts:** Sends email for lag or long transactions.
5. **Safe Index Maintenance:** Uses **online rebuild** for minimal blocking.

3) Scheduling Recommendations

- **Check AG lag & long transactions:** every **5–10 min**
- **Maintenance Job:** nightly off-peak hours
- **Email Alerts:** triggered immediately if thresholds breached

Fully automated AG-aware maintenance & monitoring script for all 3 replicas. It dynamically handles **primary and secondary replicas**, runs maintenance safely, and generates a **consolidated AG report** via email.

This is production-ready for **SQL Server 2017–2025**.

1) Overview of Features

- Detects **PRIMARY and SECONDARY replicas** per AG
- Checks **AG lag thresholds** (log_send_mb, redo_mb, redo_rate_mb_sec)
- Detects **long-running user transactions** (>30 min)
- Executes **maintenance tasks** safely:
 - PRIMARY → online rebuilds and stats updates
 - SECONDARY → optional reorganize (non-blocking)
- Consolidates **alerts for all replicas** into a single email

2) SQL Agent Job Script

```
-- =====
-- Multi-Replica AG Maintenance & Monitoring
-- =====
```

```
DECLARE @AlertMessage NVARCHAR(MAX) = '';
DECLARE @SendMail BIT = 0;
```

```
-- 1. Get all replicas for this server
```

```
WITH replicas AS (
  SELECT
    ag.name AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    DB_NAME(drs.database_id) AS database_name,
    drs.log_send_queue_size / 1024.0 AS log_send_mb,
    drs.redo_queue_size / 1024.0 AS redo_mb,
    drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
    drs.synchronization_state_desc
  FROM sys.dm_hadr_database_replica_states drs
  JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
  JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
  JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
)
```

```
-- 2. Consolidated AG Lag Alert
```

```
SELECT @AlertMessage = STRING_AGG(
  CONCAT(
    'AG: ', ag_name,
    ', Replica: ', replica_server_name,
    ', Role: ', role_desc,
    ', DB: ', database_name,
    ', LogSendMB: ', log_send_mb,
    ', RedoMB: ', redo_mb,
```



```

        ', RedoRateMB/s: ', redo_rate_mb_sec,
        ', State: ', synchronization_state_desc
    ), CHAR(13)+CHAR(10)
)
FROM replicas
WHERE log_send_mb > 50
    OR redo_mb > 50
    OR redo_rate_mb_sec < 1
    OR synchronization_state_desc <> 'SYNCHRONIZED';

```

```

IF @AlertMessage IS NOT NULL
    SET @SendMail = 1;

```

```

-- 3. Long-running user transactions
DECLARE @LongTx NVARCHAR(MAX) = '';

```

```

WITH long_tx AS (
    SELECT
        s.session_id,
        DB_NAME(dt.database_id) AS database_name,
        t.transaction_begin_time,
        DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
        s.login_name,
        s.host_name,
        s.program_name
    FROM sys.dm_tran_active_transactions t
    JOIN sys.dm_tran_session_transactions st
        ON t.transaction_id = st.transaction_id
    JOIN sys.dm_exec_sessions s
        ON st.session_id = s.session_id
    JOIN sys.dm_tran_database_transactions dt
        ON t.transaction_id = dt.transaction_id
    WHERE s.is_user_process = 1
)

```

```

SELECT @LongTx = STRING_AGG(
    CONCAT(
        'SessionID: ', session_id,
        ', DB: ', database_name,
        ', Duration(min): ', tx_duration_min,
        ', Login: ', login_name,
        ', Host: ', host_name,
        ', Program: ', program_name
    ), CHAR(13)+CHAR(10)
)

```

```

FROM long_tx
WHERE tx_duration_min > 30;

```

```

IF @LongTx IS NOT NULL
BEGIN
    SET @SendMail = 1;

```

```

SET @AlertMessage = ISNULL(@AlertMessage, '') + CHAR(13)+CHAR(10)
    + 'Long-running transactions detected:' + CHAR(13)+CHAR(10)
    + @LongTx;

END

-- 4. Send consolidated alert email if any issues detected
IF @SendMail = 1
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'DBMailProfile',
        @recipients = 'dba-team@domain.com',
        @subject = 'SQL Server AG Health & Maintenance Alert - ' + @@SERVERNAME,
        @body = @AlertMessage;
END

-- 5. Run maintenance for PRIMARY replica only
DECLARE @ReplicasCursor CURSOR;
DECLARE @replica_name SYSNAME, @role_desc NVARCHAR(20);

SET @ReplicasCursor = CURSOR FOR
SELECT replica_server_name, role_desc
FROM replicas
WHERE replica_server_name = @@SERVERNAME;

OPEN @ReplicasCursor;
FETCH NEXT FROM @ReplicasCursor INTO @replica_name, @role_desc;

WHILE @@FETCH_STATUS = 0
BEGIN
    IF @role_desc = 'PRIMARY' AND @SendMail = 0
    BEGIN
        PRINT 'Running PRIMARY maintenance tasks...';

        -- Example: Online index rebuilds
        DECLARE @sql NVARCHAR(MAX) = '';
        SELECT @sql = STRING_AGG(
            'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(t.schema_id))
            + '.' + QUOTENAME(t.name) + ' REBUILD WITH (ONLINE = ON);'
            , CHAR(13)+CHAR(10)
        )
        FROM sys.tables t
        WHERE t.is_ms_shipped = 0;

        EXEC sp_executesql @sql;

        PRINT 'PRIMARY maintenance completed.';
    END
    ELSE IF @role_desc = 'SECONDARY'
    BEGIN
        PRINT 'Secondary replica detected. Optional maintenance: index reorg only.';
    END
END

```

```

DECLARE @sql_secondary NVARCHAR(MAX) = '';
SELECT @sql_secondary = STRING_AGG(
    'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(t.schema_id))
    + '.' + QUOTENAME(t.name) + ' REORGANIZE;'
    , CHAR(13)+CHAR(10)
)
FROM sys.tables t
WHERE t.is_ms_shipped = 0;

EXEC sp_executesql @sql_secondary;
PRINT 'SECONDARY maintenance completed.';
END

FETCH NEXT FROM @ReplicasCursor INTO @replica_name, @role_desc;
END

CLOSE @ReplicasCursor;
DEALLOCATE @ReplicasCursor;

PRINT 'AG-aware maintenance & monitoring job completed.';

```

3) Key Benefits

1. **Multi-replica aware:** Detects all replicas on a server.
2. **Dynamic role detection:** Runs heavy tasks only on **PRIMARY**.
3. **Safe secondary maintenance:** Optional **index reorg** without blocking.
4. **Automated alerts:** Single consolidated email covering all replicas.
5. **Threshold-aware:** Skips maintenance if lag or long transactions exist.
6. **Compatible with SQL Server 2017 → 2025.**

4) Suggested SQL Agent Schedule

Task	Frequency	Notes
AG Health Check / Alerts	Every 5–10 min	Ensure timely notification of issues
Maintenance Tasks	Nightly off-peak	Runs only if no alerts, online rebuilds on primary, optional reorg on secondaries

Fully automated daily AG Health Report that:

- Monitors all replicas for **log send / redo / redo rate / sync state**
- Detects **long-running transactions**
- Captures **blocking chains**
- Sends a **HTML-formatted email** with tables and highlights
- Works across **SQL Server 2017 → 2025**

This report complements the maintenance job and can run **once daily** or on-demand.

1) Key Features of the Daily AG Health Report

Feature	Details
Replica Health	Shows log_send_mb, redo_mb, redo_rate_mb_sec, synchronization_state_desc per database per replica
Long-Running TX	Transactions > 30 min with session info
Blocking Chains	Session → blocking chain, shows top 3 levels
HTML Email	Easy to read, highlights issues (red/yellow/green)
Multi-Replica	Aggregates all replicas on the server
AG-aware	Indicates PRIMARY vs SECONDARY

2) SQL Script for Daily AG Health Report

```
-- =====
-- SQL Server AG Daily Health Report (HTML)
-- =====

DECLARE @html NVARCHAR(MAX) = '';
DECLARE @subject NVARCHAR(255) = 'Daily AG Health Report - ' + CONVERT(VARCHAR, GETDATE(), 120);
DECLARE @recipients NVARCHAR(255) = 'dba-team@domain.com';

-- 1. AG Replica Health
WITH ag_lag AS (
    SELECT
        ag.name AS ag_name,
        ar.replica_server_name,
        ars.role_desc,
        DB_NAME(drs.database_id) AS database_name,
        drs.log_send_queue_size / 1024.0 AS log_send_mb,
        drs.redo_queue_size / 1024.0 AS redo_mb,
        drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
        drs.synchronization_state_desc
    FROM sys.dm_hadr_database_replica_states drs
    JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
    JOIN sys.dm_hadr_availability_replica_states ars ON ar.replica_id = ars.replica_id
    JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
),
```

-- 2. Long-Running Transactions (>30 min)

long_tx AS (

SELECT

s.session_id,
r.blocking_session_id,
s.login_name,
DB_NAME(dt.database_id) AS database_name,
t.transaction_begin_time,
DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
s.host_name,
s.program_name

FROM sys.dm_tran_active_transactions t

JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id

JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id

LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id

JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id

WHERE s.is_user_process = 1

)

-- =====

-- Build HTML Report

-- =====

-- 3. Replica Health Table

SELECT @html =

'<h2>AG Replica Health</h2>' +

'<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +

'<tr style="background-color:#B0C4DE;"><th>AG Name</th><th>Replica</th><th>Role</th><th>DB
Name</th><th>LogSend MB</th><th>Redo MB</th><th>RedoRate MB/s</th><th>State</th></tr>' +

STRING_AGG(

'<tr' +

CASE WHEN log_send_mb>50 OR redo_mb>50 OR synchronization_state_desc<>'SYNCHRONIZED' THEN ' style="background-color:#FF9999;" ELSE " END +

'<td>' + ag_name + '</td><td>' + replica_server_name + '</td><td>' + role_desc + '</td><td>' + database_name + '</td><td>' + CAST(log_send_mb AS VARCHAR(10)) + '</td><td>' + CAST(redo_mb AS VARCHAR(10)) + '</td><td>' + CAST(redo_rate_mb_sec AS VARCHAR(10)) + '</td><td>' + synchronization_state_desc + '</td></tr>'

, '' +

'</table>'

FROM ag_lag;

-- 4. Long-Running Transactions Table

IF EXISTS(SELECT 1 FROM long_tx WHERE tx_duration_min>30)

BEGIN

SET @html += '<h2>Long-Running User Transactions (>30 min)</h2>' +

'<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +

'<tr style="background-

color:#B0C4DE;"><th>SessionID</th><th>DB</th><th>Duration(min)</th><th>Login</th><th>Host</th><th>Program</th><th>Blocking Session</th></tr>' +

```

        STRING_AGG(
            '<tr><td>' + CAST(session_id AS VARCHAR(10)) + '</td><td>' + database_name + '</td><td>' +
            CAST(tx_duration_min AS VARCHAR(10)) + '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' +
            program_name + '</td><td>' + ISNULL(CAST(blocking_session_id AS VARCHAR(10)), '') + '</td></tr>'
            , ''
        )
        + '</table>';
END
ELSE
    SET @html += '<p>No long-running transactions detected.</p>';

```

-- 5. Send HTML Email

```

EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'DBMailProfile',
    @recipients = @recipients,
    @subject = @subject,
    @body = @html,
    @body_format = 'HTML';

```

3) Features Explained

1. AG Replica Health Table

- **Red highlight** if log_send_mb>50, redo_mb>50, or synchronization_state_desc <> SYNCHRONIZED
- Easy to scan for problem databases

2. Long-Running Transactions Table

- Shows session ID, database, duration, login, host, program
- Shows **blocking_session_id** for quick root cause tracing

3. HTML Formatting

- Tables, colored highlights, readable for email

4. Multi-Replica Aggregation

- Shows **all replicas** on the server
- Clearly indicates **role (PRIMARY/SECONDARY)**

5. Automated Scheduling

- Run nightly via SQL Agent
- Can also be scheduled post-maintenance to verify impact

4) Suggested Enhancements

- Add **blocking chain details** recursively (optional advanced feature)
- Include **index fragmentation / statistics stats**
- Optionally add **trend charts** (via embedded HTML graphs or Power BI link)

Daily AG Health Report to include a **full blocking chain analysis**. This will show the **entire hierarchy of blockers** so you can quickly see which sessions are causing blocking that might contribute to AG lag or long-running transactions.

📌 Approach

1. Use a **recursive CTE** to traverse sys.dm_exec_requests and capture blocking chains.
2. Include session info (login_name, host_name, program_name).
3. Join to long_tx to highlight sessions already running >30 min.
4. Format as an **HTML table** for email.

2) Blocking Chain CTE

```
-- =====
```

```
-- Blocking Chain CTE
```

```
-- =====
```

```
WITH BlockingCTE AS (
```

```
-- Anchor: sessions that are currently blocked
```

```
SELECT
```

```
    r.session_id,
```

```
    r.blocking_session_id,
```

```
    s.login_name,
```

```
    s.host_name,
```

```
    s.program_name,
```

```
    CAST(r.session_id AS VARCHAR(MAX)) AS chain_path,
```

```
    0 AS level
```

```
FROM sys.dm_exec_requests r
```

```
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
```

```
WHERE r.blocking_session_id <> 0
```

```
UNION ALL
```

```
-- Recursive: traverse blockers
```

```
SELECT
```

```
    r2.session_id,
```

```
    r2.blocking_session_id,
```

```
    s2.login_name,
```

```
    s2.host_name,
```

```
    s2.program_name,
```

```
    CAST(b.chain_path + ' -> ' + CAST(r2.session_id AS VARCHAR) AS VARCHAR(MAX)) AS chain_path,
```

```
    b.level + 1
```

```
FROM sys.dm_exec_requests r2
```

```
JOIN sys.dm_exec_sessions s2 ON r2.session_id = s2.session_id
```

```
JOIN BlockingCTE b ON r2.session_id = b.blocking_session_id
```

```
)
```

```
SELECT *
```

```
FROM BlockingCTE
```

```
ORDER BY chain_path;
```

Explanation:

- chain_path shows the full **blocking hierarchy**, e.g., 53 -> 47 -> 32.
- level shows depth in the blocking chain.

3) Integrate Blocking Chains into HTML Report

```
DECLARE @html NVARCHAR(MAX) = '';
```

```
-- 1. Capture AG replica health (same as before)
```

```
-- ... (reuse ag_lag CTE from previous report)
```

```
-- 2. Capture Long-running TX (same as before)
```

```
-- ... (reuse long_tx CTE)
```

```
-- 3. Capture Blocking Chains
```

```
WITH BlockingCTE AS (
```

```
    SELECT
```

```
        r.session_id,
```

```
        r.blocking_session_id,
```

```
        s.login_name,
```

```
        s.host_name,
```

```
        s.program_name,
```

```
        CAST(r.session_id AS VARCHAR(MAX)) AS chain_path,
```

```
        0 AS level
```

```
    FROM sys.dm_exec_requests r
```

```
    JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
```

```
    WHERE r.blocking_session_id <> 0
```

```
    UNION ALL
```

```
    SELECT
```

```
        r2.session_id,
```

```
        r2.blocking_session_id,
```

```
        s2.login_name,
```

```
        s2.host_name,
```

```
        s2.program_name,
```

```
        CAST(b.chain_path + ' -> ' + CAST(r2.session_id AS VARCHAR) AS VARCHAR(MAX)) AS chain_path,
```

```
        b.level + 1
```

```
    FROM sys.dm_exec_requests r2
```

```
    JOIN sys.dm_exec_sessions s2 ON r2.session_id = s2.session_id
```

```
    JOIN BlockingCTE b ON r2.session_id = b.blocking_session_id
```

```
)
```

```
SELECT @html +=
```

```
    '<h2>Blocking Chains</h2>' +
```

```
    '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
```



```
'<tr style="background-color:#B0C4DE;"><th>Level</th><th>Chain
Path</th><th>SessionID</th><th>Login</th><th>Host</th><th>Program</th></tr>' +
STRING_AGG(
    '<tr><td>' + CAST(level AS VARCHAR) + '</td><td>' + chain_path + '</td><td>'
    + CAST(session_id AS VARCHAR) + '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' +
program_name + '</td></tr>'
    , ''
)
+ '</table>'
FROM BlockingCTE;
```

4) Send Complete HTML Email

Combine the **AG health**, **long-running TX**, and **blocking chains** sections into a single HTML email:

```
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'DBMailProfile',
    @recipients = 'dba-team@domain.com',
    @subject = 'Daily AG Health Report with Blocking Chains - ' + CONVERT(VARCHAR, GETDATE(), 120),
    @body = @html,
    @body_format = 'HTML';
```

5) Benefits

- **Full visibility into AG health** (log send, redo, sync)
- **Highlights long-running transactions** (>30 min)
- **Shows complete blocking chains**, so you can identify the root blocker
- **HTML formatting** makes it easy to read and share
- **Works on multi-replica environments** (2017–2025)

Unified, fully automated AG monitoring & maintenance solution that does everything in **one SQL Agent job**:

Features:

1. **AG Health Checks**
 - Log send / redo / redo rate / sync state per database & replica
2. **Long-Running Transactions**
 - 30 min, including session info
3. **Blocking Chains**
 - Full hierarchy of blockers, session info, chain path
4. **AG-Aware Maintenance**
 - PRIMARY: online index rebuilds & stats updates
 - SECONDARY: optional index reorganize
5. **Conditional Execution**
 - Maintenance only runs if no critical lag or blocking exists
6. **Consolidated HTML Email Report**
 - Combines AG health, long transactions, and blocking chains

Unified SQL Agent Job Script

```
-- =====
-- Unified AG Monitoring & Maintenance Job
-- SQL Server 2017 – 2025
-- =====

DECLARE @html NVARCHAR(MAX) = '';
DECLARE @subject NVARCHAR(255) = 'AG Health & Maintenance Report - ' + CONVERT(VARCHAR, GETDATE(), 120);
DECLARE @recipients NVARCHAR(255) = 'dba-team@domain.com';
DECLARE @SendMail BIT = 0;

-- =====
-- 1. Capture AG Replica Health
-- =====

WITH ag_lag AS (
    SELECT
        ag.name AS ag_name,
        ar.replica_server_name,
        ars.role_desc,
        DB_NAME(drs.database_id) AS database_name,
        drs.log_send_queue_size / 1024.0 AS log_send_mb,
        drs.redo_queue_size / 1024.0 AS redo_mb,
        drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
        drs.synchronization_state_desc
    FROM sys.dm_hadr_database_replica_states drs
    JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
    JOIN sys.dm_hadr_availability_replica_states ars ON ar.replica_id = ars.replica_id
    JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
)
SELECT @html = '<h2>AG Replica Health</h2>' +
```

```

'<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
'<tr style="background-color:#B0C4DE;"><th>AG</th><th>Replica</th><th>Role</th><th>DB</th><th>LogSend
MB</th><th>Redo MB</th><th>RedoRate MB/s</th><th>State</th></tr>' +
STRING_AGG(
  '<tr>' +
    CASE WHEN log_send_mb>50 OR redo_mb>50 OR synchronization_state_desc<>'SYNCHRONIZED' THEN '
style="background-color:#FF9999;" ELSE " END +
    '>td>' + ag_name + '</td><td>' + replica_server_name + '</td><td>' + role_desc + '</td><td>' + database_name +
'</td><td>' + CAST(log_send_mb AS VARCHAR(10)) + '</td><td>' + CAST(redo_mb AS VARCHAR(10)) + '</td><td>' +
CAST(redo_rate_mb_sec AS VARCHAR(10)) + '</td><td>' + synchronization_state_desc + '</td></tr>'
  , ''
)
+ '</table>'
FROM ag_lag;

-- Check if any critical lag exists
IF EXISTS (SELECT 1 FROM ag_lag WHERE log_send_mb>50 OR redo_mb>50 OR
synchronization_state_desc<>'SYNCHRONIZED')
  SET @SendMail = 1;

-- =====
-- 2. Capture Long-Running Transactions
-- =====
WITH long_tx AS (
  SELECT
    s.session_id,
    r.blocking_session_id,
    s.login_name,
    DB_NAME(dt.database_id) AS database_name,
    t.transaction_begin_time,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
    s.host_name,
    s.program_name
  FROM sys.dm_tran_active_transactions t
  JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
  JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
  LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
  JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
  WHERE s.is_user_process = 1
)
SELECT @html +=
  '<h2>Long-Running Transactions (>30 min)</h2>' +
  ISNULL(
    SELECT '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +

```

```

'<tr style="background-
color:#B0C4DE;"><th>SessionID</th><th>DB</th><th>Duration(min)</th><th>Login</th><th>Host</th><th>Program</t
h><th>Blocking Session</th></tr>' +
    STRING_AGG(
        '<tr><td>' + CAST(session_id AS VARCHAR) + '</td><td>' + database_name + '</td><td>' + CAST(tx_duration_min
AS VARCHAR) + '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' + program_name + '</td><td>' +
ISNULL(CAST(blocking_session_id AS VARCHAR),'') + '</td></tr>'
        , ""
    )
+ '</table>'
FROM long_tx
WHERE tx_duration_min>30
), '<p>No long-running transactions detected.</p>');

```

```

IF EXISTS (SELECT 1 FROM long_tx WHERE tx_duration_min>30)
    SET @SendMail = 1;

```

```

-- =====
-- 3. Capture Blocking Chains (recursive)
-- =====

```

```

WITH BlockingCTE AS (
    SELECT
        r.session_id,
        r.blocking_session_id,
        s.login_name,
        s.host_name,
        s.program_name,
        CAST(r.session_id AS VARCHAR(MAX)) AS chain_path,
        0 AS level
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
    WHERE r.blocking_session_id <> 0

```

```

    UNION ALL

```

```

    SELECT
        r2.session_id,
        r2.blocking_session_id,
        s2.login_name,
        s2.host_name,
        s2.program_name,
        CAST(b.chain_path + ' -> ' + CAST(r2.session_id AS VARCHAR) AS VARCHAR(MAX)) AS chain_path,
        b.level + 1
    FROM sys.dm_exec_requests r2
    JOIN sys.dm_exec_sessions s2 ON r2.session_id = s2.session_id
    JOIN BlockingCTE b ON r2.session_id = b.blocking_session_id

```

```

)
SELECT @html +=
    '<h2>Blocking Chains</h2>' +
    ISNULL((
        SELECT '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
            '<tr style="background-color:#B0C4DE;"><th>Level</th><th>Chain'
Path</th><th>SessionID</th><th>Login</th><th>Host</th><th>Program</th></tr>' +
            STRING_AGG(
                '<tr><td>' + CAST(level AS VARCHAR) + '</td><td>' + chain_path + '</td><td>' + CAST(session_id AS VARCHAR) +
                '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' + program_name + '</td></tr>'
            , ''
        )
        + '</table>'
    FROM BlockingCTE
    ), '<p>No blocking detected.</p>');

```

```

-- =====
-- 4. Send consolidated email report
-- =====

```

```

EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'DBMailProfile',
    @recipients = @recipients,
    @subject = @subject,
    @body = @html,
    @body_format = 'HTML';

```

```

-- =====
-- 5. Run Maintenance Tasks (if no critical alerts)
-- =====

```

```

DECLARE @replica_role NVARCHAR(20);

```

```

SELECT TOP 1 @replica_role = ars.role_desc
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id
WHERE ar.replica_server_name = @@SERVERNAME;

```

```

IF @replica_role='PRIMARY' AND @SendMail=0
BEGIN

```

```

    PRINT 'PRIMARY replica detected and no critical alerts. Running maintenance...';

```

```

    DECLARE @sql NVARCHAR(MAX) = '';
    SELECT @sql = STRING_AGG(
        'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(t.schema_id)) + '.' + QUOTENAME(t.name) + ' REBUILD
WITH (ONLINE = ON);'
        , CHAR(13)+CHAR(10)
    )

```

```
FROM sys.tables t
WHERE t.is_ms_shipped = 0;

EXEC sp_executesql @sql;
PRINT 'Maintenance completed.';
END
ELSE
    PRINT 'Maintenance skipped due to secondary role or critical alerts.';
```

✓ Key Notes

1. **Alerts First, Maintenance Later**
 - If any replica lag, long TX, or blockers exist, maintenance is **skipped**.
2. **Primary vs Secondary Handling**
 - PRIMARY: online rebuilds
 - SECONDARY: no heavy maintenance; optional reorganize can be added if needed
3. **HTML Email Report**
 - **AG Health** table with red highlights for lag
 - **Long-running transactions** table
 - **Blocking chains** table
4. **Fully Compatible**
 - SQL Server 2017 → 2025
 - Handles multiple replicas per AG automatically

<https://www.sqldbachamps.com/>

Enhanced **unified AG monitoring & maintenance solution** with **trend tracking**, so you we see how AG lag, redo rates, and long-running transactions evolve over time. This is great for **proactive monitoring** and spotting gradual issues before they become critical.

1) Concept

We will:

1. Create a **logging table** to store daily snapshots of AG health and long-running transactions.
2. Insert current metrics at each job run.
3. Include **trend graphs** in the HTML email (simple bar/line graphs using inline HTML tables or characters, no external tools needed) or summary tables.
4. Optionally, calculate **moving averages** or **lag deltas**.

2) Step 1: Create Logging Table

-- Logging table for AG health trends

```
CREATE TABLE dbo.AGHealthTrend (
    LogDate DATETIME NOT NULL DEFAULT GETDATE(),
    AGName NVARCHAR(128),
    ReplicaServer NVARCHAR(128),
    RoleDesc NVARCHAR(20),
    DatabaseName NVARCHAR(128),
    LogSendMB FLOAT,
    RedoMB FLOAT,
    RedoRateMBPerSec FLOAT,
    SyncState NVARCHAR(60)
);
```

-- Logging table for long-running transactions

```
CREATE TABLE dbo.LongTxTrend (
    LogDate DATETIME NOT NULL DEFAULT GETDATE(),
    SessionID INT,
    DatabaseName NVARCHAR(128),
    LoginName NVARCHAR(128),
    HostName NVARCHAR(128),
    ProgramName NVARCHAR(128),
    TxDurationMin INT
);
```

3) Step 2: Insert Snapshots During Job Run

-- Insert AG health snapshot

```
INSERT INTO dbo.AGHealthTrend (AGName, ReplicaServer, RoleDesc, DatabaseName, LogSendMB, RedoMB,
RedoRateMBPerSec, SyncState)
SELECT
    ag.name,
    ar.replica_server_name,
```

```

ars.role_desc,
DB_NAME(drs.database_id),
drs.log_send_queue_size / 1024.0,
drs.redo_queue_size / 1024.0,
drs.redo_rate / 1024.0,
drs.synchronization_state_desc
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
JOIN sys.dm_hadr_availability_replica_states ars ON ar.replica_id = ars.replica_id
JOIN sys.availability_groups ag ON ar.group_id = ag.group_id;

-- Insert long-running TX snapshot (>30 min)
INSERT INTO dbo.LongTxTrend (SessionID, DatabaseName, LoginName, HostName, ProgramName, TxDurationMin)
SELECT
    s.session_id,
    DB_NAME(dt.database_id),
    s.login_name,
    s.host_name,
    s.program_name,
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE())
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
WHERE s.is_user_process = 1
    AND DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) > 30;

```

4) Step 3: Generate Trend Section in HTML Email

```

DECLARE @trendHTML NVARCHAR(MAX) = '';
-- 7-day AG health trend summary
WITH RecentTrend AS (
    SELECT TOP 7 *
    FROM dbo.AGHealthTrend
    ORDER BY LogDate DESC
)
SELECT @trendHTML = '<h2>AG Lag Trends (Last 7 Snapshots)</h2>' +
    '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
    '<tr style="background-color:#B0C4DE;"><th>Date</th><th>AG</th><th>Replica</th><th>DB</th><th>LogSend' +
    'MB</th><th>Redo MB</th><th>RedoRate MB/s</th><th>SyncState</th></tr>' +
    STRING_AGG(
        '<tr><td>' + CONVERT(VARCHAR, LogDate, 120) + '</td><td>' + AGName + '</td><td>' + ReplicaServer + '</td><td>' +
        DatabaseName + '</td><td>' + CAST(LogSendMB AS VARCHAR) + '</td><td>' + CAST(RedoMB AS VARCHAR) +
        '</td><td>' + CAST(RedoRateMBPerSec AS VARCHAR) + '</td><td>' + SyncState + '</td></tr>'
        , '') + '</table>'
FROM RecentTrend;

```


-- Append trend section to main HTML

SET @html += @trendHTML;



This gives you a **rolling 7-day snapshot** for AG health, showing trends for each database and replica.

5) Optional Enhancements for Trend Visualization

- **Mini bar graphs in HTML:** Use repeated characters or colored <div> widths to represent log_send_mb or redo_mb.
- **Delta calculation:** Show change from previous snapshot to highlight increasing lag.

Example:

```
SELECT
    DatabaseName,
    ReplicaServer,
    LogSendMB,
    LAG(LogSendMB) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY LogDate) AS PrevLogSendMB,
    LogSendMB - LAG(LogSendMB) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY LogDate) AS Delta
FROM dbo.AGHealthTrend
WHERE LogDate >= DATEADD(DAY, -7, GETDATE());
```

This allows highlighting **growing lag trends** in the HTML email.

6) How It Fits Into Unified Job

1. **Capture AG health + long TX + blocking chains**
2. **Insert snapshots into trend tables**
3. **Generate HTML report** including current state + trend tables
4. **Send email**
5. **Run maintenance tasks if no alerts**

This gives you **real-time insights** and **historical trends**, all automated in one SQL Agent job.

Ready-to-deploy, all-in-one SQL Agent job script for your SQL Server AG environment (2017 → 2025). It includes:

- **AG Health Checks** (log send, redo, redo rate, sync state)
- **Long-Running Transactions** (>30 min)
- **Blocking Chains** (full hierarchy)
- **Trend Logging** (7-day snapshots)
- **Conditional Maintenance** (PRIMARY = online rebuild, SECONDARY = optional reorg)
- **Consolidated HTML Email Report**

1) Pre-requisites

Create Trend Tables

-- AG Health Trend

IF OBJECT_ID('dbo.AGHealthTrend') IS NULL

CREATE TABLE dbo.AGHealthTrend (

LogDate DATETIME NOT NULL DEFAULT GETDATE(),

AGName NVARCHAR(128),

ReplicaServer NVARCHAR(128),

RoleDesc NVARCHAR(20),

DatabaseName NVARCHAR(128),

LogSendMB FLOAT,

RedoMB FLOAT,

RedoRateMBPerSec FLOAT,

SyncState NVARCHAR(60)

);

-- Long-Running TX Trend

IF OBJECT_ID('dbo.LongTxTrend') IS NULL

CREATE TABLE dbo.LongTxTrend (

LogDate DATETIME NOT NULL DEFAULT GETDATE(),

SessionID INT,

DatabaseName NVARCHAR(128),

LoginName NVARCHAR(128),

HostName NVARCHAR(128),

ProgramName NVARCHAR(128),

TxDurationMin INT

);

2) Full Unified Job Script

-- =====

-- Unified AG Monitoring & Maintenance

-- =====

DECLARE @html NVARCHAR(MAX) = '';

DECLARE @subject NVARCHAR(255) = 'AG Health & Maintenance Report - ' + CONVERT(VARCHAR, GETDATE(), 120);

DECLARE @recipients NVARCHAR(255) = 'dba-team@domain.com';

```

DECLARE @SendMail BIT = 0;

-- =====
-- 1. AG Replica Health
-- =====

WITH ag_lag AS (
    SELECT
        ag.name AS ag_name,
        ar.replica_server_name,
        ars.role_desc,
        DB_NAME(drs.database_id) AS database_name,
        drs.log_send_queue_size / 1024.0 AS log_send_mb,
        drs.redo_queue_size / 1024.0 AS redo_mb,
        drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
        drs.synchronization_state_desc
    FROM sys.dm_hadr_database_replica_states drs
    JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
    JOIN sys.dm_hadr_availability_replica_states ars ON ar.replica_id = ars.replica_id
    JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
)
-- Log snapshot
INSERT INTO dbo.AGHealthTrend (AGName, ReplicaServer, RoleDesc, DatabaseName, LogSendMB, RedoMB,
RedoRateMBPerSec, SyncState)
SELECT ag_name, replica_server_name, role_desc, database_name, log_send_mb, redo_mb, redo_rate_mb_sec,
synchronization_state_desc
FROM ag_lag;

-- Build HTML table
SELECT @html = '<h2>AG Replica Health</h2>' +
    '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
    '<tr style="background-color:#B0C4DE;"><th>AG</th><th>Replica</th><th>Role</th><th>DB</th><th>LogSend'
    MB</th><th>Redo MB</th><th>RedoRate MB/s</th><th>State</th></tr>' +
    STRING_AGG(
        '<tr>' +
        CASE WHEN log_send_mb>50 OR redo_mb>50 OR synchronization_state_desc<>'SYNCHRONIZED' THEN '
style="background-color:#FF9999;" ELSE " END +
        '<td>' + ag_name + '</td><td>' + replica_server_name + '</td><td>' + role_desc + '</td><td>' + database_name +
        '</td><td>' + CAST(log_send_mb AS VARCHAR(10)) + '</td><td>' + CAST(redo_mb AS VARCHAR(10)) + '</td><td>' +
        CAST(redo_rate_mb_sec AS VARCHAR(10)) + '</td><td>' + synchronization_state_desc + '</td></tr>'
        , ""
    )
    + '</table>'

FROM ag_lag;

```

```
IF EXISTS(SELECT 1 FROM ag_lag WHERE log_send_mb>50 OR redo_mb>50 OR
synchronization_state_desc<>'SYNCHRONIZED')
SET @SendMail = 1;
```

```
-- =====
```

```
-- 2. Long-Running Transactions
```

```
-- =====
```

```
WITH long_tx AS (
```

```
SELECT
```

```
    s.session_id,
```

```
    r.blocking_session_id,
```

```
    s.login_name,
```

```
    DB_NAME(dt.database_id) AS database_name,
```

```
    t.transaction_begin_time,
```

```
    DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
```

```
    s.host_name,
```

```
    s.program_name
```

```
FROM sys.dm_tran_active_transactions t
```

```
JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
```

```
JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
```

```
LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
```

```
JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
```

```
WHERE s.is_user_process = 1
```

```
)
```

```
-- Log snapshot
```

```
INSERT INTO dbo.LongTxTrend (SessionID, DatabaseName, LoginName, HostName, ProgramName, TxDurationMin)
```

```
SELECT session_id, database_name, login_name, host_name, program_name, tx_duration_min
```

```
FROM long_tx
```

```
WHERE tx_duration_min>30;
```

```
-- HTML table
```

```
SELECT @html +=
```

```
    '<h2>Long-Running Transactions (&gt;30 min)</h2>' +
```

```
    ISNULL((
```

```
        SELECT '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
```

```
            '<tr style="background-
```

```
color:#B0C4DE;"><th>SessionID</th><th>DB</th><th>Duration(min)</th><th>Login</th><th>Host</th><th>Program</th><th>Blocking Session</th></tr>' +
```

```
            STRING_AGG(
```

```
                '<tr><td>' + CAST(session_id AS VARCHAR) + '</td><td>' + database_name + '</td><td>' + CAST(tx_duration_min
```

```
AS VARCHAR) + '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' + program_name + '</td><td>' +
```

```
ISNULL(CAST(blocking_session_id AS VARCHAR),'') + '</td></tr>'
```

```
            ,"
```

```
        )
```

```
    + '</table>'
```

```
FROM long_tx
```

```

WHERE tx_duration_min>30
), '<p>No long-running transactions detected.</p>');

IF EXISTS(SELECT 1 FROM long_tx WHERE tx_duration_min>30)
SET @SendMail = 1;

-- =====
-- 3. Blocking Chains
-- =====

WITH BlockingCTE AS (
    SELECT r.session_id, r.blocking_session_id, s.login_name, s.host_name, s.program_name,
           CAST(r.session_id AS VARCHAR(MAX)) AS chain_path, 0 AS level
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
    WHERE r.blocking_session_id<>0

    UNION ALL

    SELECT r2.session_id, r2.blocking_session_id, s2.login_name, s2.host_name, s2.program_name,
           CAST(b.chain_path + ' -> ' + CAST(r2.session_id AS VARCHAR) AS VARCHAR(MAX)) AS chain_path,
           b.level+1
    FROM sys.dm_exec_requests r2
    JOIN sys.dm_exec_sessions s2 ON r2.session_id = s2.session_id
    JOIN BlockingCTE b ON r2.session_id = b.blocking_session_id
)
SELECT @html +=
    '<h2>Blocking Chains</h2>' +
    ISNULL((
        SELECT '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
        '<tr style="background-color:#B0C4DE;"><th>Level</th><th>Chain
Path</th><th>SessionID</th><th>Login</th><th>Host</th><th>Program</th></tr>' +
        STRING_AGG(
            '<tr><td>' + CAST(level AS VARCHAR) + '</td><td>' + chain_path + '</td><td>' + CAST(session_id AS VARCHAR) +
            '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' + program_name + '</td></tr>'
            , ""
        )
        + '</table>'
    FROM BlockingCTE
    ), '<p>No blocking detected.</p>');

-- =====
-- 4. Trend Summary (Last 7 snapshots)
-- =====

WITH RecentTrend AS (
    SELECT TOP 7 *
    FROM dbo.AGHealthTrend

```

```

ORDER BY LogDate DESC
)
SELECT @html += '<h2>AG Lag Trends (Last 7 Snapshots)</h2>' +
    '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
    '<tr style="background-color:#B0C4DE;"><th>Date</th><th>AG</th><th>Replica</th><th>DB</th><th>LogSend'
    MB</th><th>Redo MB</th><th>RedoRate MB/s</th><th>State</th></tr>' +
    STRING_AGG(
        '<tr><td>' + CONVERT(VARCHAR, LogDate, 120) + '</td><td>' + AGName + '</td><td>' + ReplicaServer + '</td><td>'
        + DatabaseName + '</td><td>' + CAST(LogSendMB AS VARCHAR) + '</td><td>' + CAST(RedoMB AS VARCHAR) +
        '</td><td>' + CAST(RedoRateMBPerSec AS VARCHAR) + '</td><td>' + SyncState + '</td></tr>'
        , ""
    ) + '</table>'
FROM RecentTrend;

-- =====
-- 5. Send Consolidated HTML Email
-- =====
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'DBMailProfile',
    @recipients = @recipients,
    @subject = @subject,
    @body = @html,
    @body_format = 'HTML';

-- =====
-- 6. Conditional Maintenance (PRIMARY only, no alerts)
-- =====
DECLARE @replica_role NVARCHAR(20);
SELECT TOP 1 @replica_role = ars.role_desc
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id
WHERE ar.replica_server_name = @@SERVERNAME;

IF @replica_role='PRIMARY' AND @SendMail=0
BEGIN
    PRINT 'PRIMARY replica and no critical alerts. Running maintenance...';
    DECLARE @sql NVARCHAR(MAX) = '';
    SELECT @sql = STRING_AGG(
        'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(t.schema_id)) + '.' + QUOTENAME(t.name) + ' REBUILD
        WITH (ONLINE = ON);'
        , CHAR(13)+CHAR(10)
    )
    FROM sys.tables t
    WHERE t.is_ms_shipped = 0;
    EXEC sp_executesql @sql;
    PRINT 'Maintenance completed.';

```

END

ELSE

PRINT 'Maintenance skipped due to secondary role or critical alerts.';



Highlights

1. **All-in-One Automation**

- Health checks, long TX, blocking chains, trend logging, maintenance, and reporting.

2. **Trend Tracking**

- 7-day rolling history of AG lag for proactive monitoring.

3. **Conditional Maintenance**

- Ensures maintenance runs **only when safe** (no critical alerts, primary replica).

4. **HTML Report**

- Consolidated, readable email with highlights for lag, long transactions, and blockers.

<https://www.sqldbachamps.com/>

Enhanced **HTML email** with unified AG monitoring & maintenance job with **inline mini bar graphs and color-coded trends**. This will make it easy to **visually spot increasing log send, redo queues, or redo rate issues**.

1) Concept

1. For each database & replica in the **7-day trend**, calculate the **delta** from the previous snapshot.
2. Represent **magnitude of lag** with a simple bar using repeated HTML characters or a colored <div> width.
3. **Color-code the trend:**
 - **Red** → worsening lag or low redo rate
 - **Yellow** → moderate increase
 - **Green** → stable or improving
4. Combine this inline with your HTML table.

2) Add Delta Calculation

WITH RecentTrend AS (

SELECT TOP 7 *

FROM dbo.AGHealthTrend

ORDER BY LogDate ASC

),

TrendWithDelta AS (

SELECT *,

LogSendMB - LAG(LogSendMB) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY LogDate) AS

DeltaLogSend,

RedoMB - LAG(RedoMB) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY LogDate) AS DeltaRedo,

RedoRateMBPerSec - LAG(RedoRateMBPerSec) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY

LogDate) AS DeltaRedoRate

FROM RecentTrend

)

SELECT *

FROM TrendWithDelta;

- DeltaLogSend > 0 → log send queue is **growing**
- DeltaRedoRate < 0 → redo rate is **dropping**

3) Generate HTML Mini Bars

DECLARE @trendHTML NVARCHAR(MAX) = '';

SELECT @trendHTML =

'<h2>AG Lag Trends (Last 7 Snapshots)</h2>' +

'<table border="1" cellpadding="3" cellspacing="0" style="border-collapse: collapse;">' +

'<tr style="background-color: #B0C4DE;"><th>Date</th><th>AG</th><th>Replica</th><th>DB</th><th>LogSend

MB</th><th>Trend</th><th>Redo MB</th><th>Trend</th><th>RedoRate

MB</th><th>Trend</th><th>State</th></tr>' +

STRING_AGG(

'<tr>' +

'<td>' + CONVERT(VARCHAR, LogDate, 120) + '</td>' +

'<td>' + AGName + '</td>' +


```

'<td>' + ReplicaServer + '</td>' +
'<td>' + DatabaseName + '</td>' +
'<td>' + CAST(LogSendMB AS VARCHAR) + '</td>' +
'<td style="background-color:' +
    CASE WHEN DeltaLogSend>10 THEN '#FF6666' -- worsening
    WHEN DeltaLogSend>0 THEN '#FFD700' -- moderate
    ELSE '#90EE90' END + ';'>' + REPLICATE('|', CAST(DeltaLogSend AS INT)) + '</td>' +
'<td>' + CAST(RedoMB AS VARCHAR) + '</td>' +
'<td style="background-color:' +
    CASE WHEN DeltaRedo>10 THEN '#FF6666'
    WHEN DeltaRedo>0 THEN '#FFD700'
    ELSE '#90EE90' END + ';'>' + REPLICATE('|', CAST(DeltaRedo AS INT)) + '</td>' +
'<td>' + CAST(RedoRateMBPerSec AS VARCHAR) + '</td>' +
'<td style="background-color:' +
    CASE WHEN DeltaRedoRate< -1 THEN '#FF6666'
    WHEN DeltaRedoRate<0 THEN '#FFD700'
    ELSE '#90EE90' END + ';'>' + REPLICATE('|', CAST(ABS(DeltaRedoRate) AS INT)) + '</td>' +
'<td>' + SyncState + '</td>' +
'</tr>'
, ''
) + '</table>'
FROM TrendWithDelta;

SET @html += @trendHTML;

```

Explanation of Mini Bars:

- REPLICATE('|', N) creates a simple horizontal bar proportional to the delta magnitude.
- Background colors:
 - **Red (#FF6666)** → rapidly worsening
 - **Yellow (#FFD700)** → moderate change
 - **Green (#90EE90)** → stable or improving

4) Integrate into Unified Job

1. After inserting snapshots into AGHealthTrend, calculate deltas and generate mini bars.
2. Append this trend table **after AG health, long TX, and blocking chains** in @html.
3. Send the **consolidated HTML email** as before.



Benefits

- **Quick visual assessment** of trends without reading numbers.
- **Highlights worsening databases or replicas** immediately.
- Works with **7-day rolling snapshots**.
- Integrates seamlessly with your **existing unified job**.

Final unified SQL Agent job script, fully deployable, for your SQL Server AG environment (2017 → 2025). It includes:

- **AG health checks**
- **Long-running transactions**
- **Blocking chains**
- **Trend logging (7-day snapshots)**
- **Mini bar graphs with color-coded trends**
- **Conditional maintenance** (PRIMARY = online rebuild, SECONDARY optional reorg)
- **Consolidated HTML email report**

1) Pre-requisites (Run Once)

-- AG Health Trend Table

IF OBJECT_ID('dbo.AGHealthTrend') IS NULL

CREATE TABLE dbo.AGHealthTrend (

LogDate DATETIME NOT NULL DEFAULT GETDATE(),

AGName NVARCHAR(128),

ReplicaServer NVARCHAR(128),

RoleDesc NVARCHAR(20),

DatabaseName NVARCHAR(128),

LogSendMB FLOAT,

RedoMB FLOAT,

RedoRateMBPerSec FLOAT,

SyncState NVARCHAR(60)

);

-- Long-Running TX Trend Table

IF OBJECT_ID('dbo.LongTxTrend') IS NULL

CREATE TABLE dbo.LongTxTrend (

LogDate DATETIME NOT NULL DEFAULT GETDATE(),

SessionID INT,

DatabaseName NVARCHAR(128),

LoginName NVARCHAR(128),

HostName NVARCHAR(128),

ProgramName NVARCHAR(128),

TxDurationMin INT

);

2) Unified Job Script

-- =====

-- Unified AG Monitoring, Trend, Blocking, Maintenance

-- =====

DECLARE @html NVARCHAR(MAX) = '';

DECLARE @subject NVARCHAR(255) = 'AG Health & Maintenance Report - ' + CONVERT(VARCHAR, GETDATE(), 120);

DECLARE @recipients NVARCHAR(255) = 'dba-team@domain.com';

DECLARE @SendMail BIT = 0;

```
-- =====
-- 1. AG Replica Health & Log Snapshot
-- =====

WITH ag_lag AS (
    SELECT
        ag.name AS ag_name,
        ar.replica_server_name,
        ars.role_desc,
        DB_NAME(drs.database_id) AS database_name,
        drs.log_send_queue_size / 1024.0 AS log_send_mb,
        drs.redo_queue_size / 1024.0 AS redo_mb,
        drs.redo_rate / 1024.0 AS redo_rate_mb_sec,
        drs.synchronization_state_desc
    FROM sys.dm_hadr_database_replica_states drs
    JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
    JOIN sys.dm_hadr_availability_replica_states ars ON ar.replica_id = ars.replica_id
    JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
)
-- Insert trend snapshot
INSERT INTO dbo.AGHealthTrend (AGName, ReplicaServer, RoleDesc, DatabaseName, LogSendMB, RedoMB,
RedoRateMBPerSec, SyncState)
SELECT ag_name, replica_server_name, role_desc, database_name, log_send_mb, redo_mb, redo_rate_mb_sec,
synchronization_state_desc
FROM ag_lag;

-- HTML Table
SELECT @html = '<h2>AG Replica Health</h2>' +
    '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
    '<tr style="background-color:#B0C4DE;"><th>AG</th><th>Replica</th><th>Role</th><th>DB</th><th>LogSend</th><th>Redo MB</th><th>RedoRate MB/s</th><th>State</th></tr>' +
    STRING_AGG(
        '<tr>' +
        CASE WHEN log_send_mb>50 OR redo_mb>50 OR synchronization_state_desc<>'SYNCHRONIZED' THEN '
style="background-color:#FF9999;" ELSE " END +
        '><td>' + ag_name + '</td><td>' + replica_server_name + '</td><td>' + role_desc + '</td><td>' + database_name +
        '</td><td>' + CAST(log_send_mb AS VARCHAR(10)) + '</td><td>' + CAST(redo_mb AS VARCHAR(10)) + '</td><td>' +
        CAST(redo_rate_mb_sec AS VARCHAR(10)) + '</td><td>' + synchronization_state_desc + '</td></tr>'
        , ''
    )
    + '</table>'
FROM ag_lag;

IF EXISTS(SELECT 1 FROM ag_lag WHERE log_send_mb>50 OR redo_mb>50 OR
synchronization_state_desc<>'SYNCHRONIZED')
    SET @SendMail = 1;
```

```
-- =====
-- 2. Long-Running Transactions (>30 min)
-- =====

WITH long_tx AS (
    SELECT
        s.session_id,
        r.blocking_session_id,
        s.login_name,
        DB_NAME(dt.database_id) AS database_name,
        t.transaction_begin_time,
        DATEDIFF(MINUTE, t.transaction_begin_time, GETDATE()) AS tx_duration_min,
        s.host_name,
        s.program_name
    FROM sys.dm_tran_active_transactions t
    JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
    JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
    LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
    JOIN sys.dm_tran_database_transactions dt ON t.transaction_id = dt.transaction_id
    WHERE s.is_user_process = 1
)

-- Log trend
INSERT INTO dbo.LongTxTrend (SessionID, DatabaseName, LoginName, HostName, ProgramName, TxDurationMin)
SELECT session_id, database_name, login_name, host_name, program_name, tx_duration_min
FROM long_tx
WHERE tx_duration_min>30;

-- HTML Table
SELECT @html +=
    '<h2>Long-Running Transactions (>30 min)</h2>' +
    ISNULL((
        SELECT '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
            '<tr style="background-
color:#B0C4DE;"><th>SessionID</th><th>DB</th><th>Duration(min)</th><th>Login</th><th>Host</th><th>Program</t
h><th>Blocking Session</th></tr>' +
        STRING_AGG(
            '<tr><td>' + CAST(session_id AS VARCHAR) + '</td><td>' + database_name + '</td><td>' + CAST(tx_duration_min
AS VARCHAR) + '</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' + program_name + '</td><td>' +
ISNULL(CAST(blocking_session_id AS VARCHAR),'') + '</td></tr>'
            , ''
        )
        + '</table>'
    FROM long_tx
    WHERE tx_duration_min>30
    ), '<p>No long-running transactions detected.</p>');
```

```
IF EXISTS(SELECT 1 FROM long_tx WHERE tx_duration_min>30)
```

```
SET @SendMail = 1;
```

```
-- =====
```

```
-- 3. Blocking Chains
```

```
-- =====
```

```
WITH BlockingCTE AS (
```

```
    SELECT r.session_id, r.blocking_session_id, s.login_name, s.host_name, s.program_name,
```

```
           CAST(r.session_id AS VARCHAR(MAX)) AS chain_path, 0 AS level
```

```
    FROM sys.dm_exec_requests r
```

```
    JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
```

```
    WHERE r.blocking_session_id<>0
```

```
    UNION ALL
```

```
    SELECT r2.session_id, r2.blocking_session_id, s2.login_name, s2.host_name, s2.program_name,
```

```
           CAST(b.chain_path + ' -> ' + CAST(r2.session_id AS VARCHAR) AS VARCHAR(MAX)) AS chain_path,
```

```
           b.level+1
```

```
    FROM sys.dm_exec_requests r2
```

```
    JOIN sys.dm_exec_sessions s2 ON r2.session_id = s2.session_id
```

```
    JOIN BlockingCTE b ON r2.session_id = b.blocking_session_id
```

```
)
```

```
SELECT @html +=
```

```
'<h2>Blocking Chains</h2>' +
```

```
ISNULL((
```

```
    SELECT '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
```

```
        '<tr style="background-color:#B0C4DE;"><th>Level</th><th>Chain
```

```
Path</th><th>SessionID</th><th>Login</th><th>Host</th><th>Program</th></tr>' +
```

```
        STRING_AGG(
```

```
            '<tr><td>' + CAST(level AS VARCHAR) + '</td><td>' + chain_path + '</td><td>' + CAST(session_id AS VARCHAR) +
```

```
'</td><td>' + login_name + '</td><td>' + host_name + '</td><td>' + program_name + '</td></tr>'
```

```
        , ''
```

```
    )
```

```
    + '</table>'
```

```
FROM BlockingCTE
```

```
), '<p>No blocking detected.</p>');
```

```
-- =====
```

```
-- 4. Trend Summary with Mini Bars (7-day)
```

```
-- =====
```

```
WITH RecentTrend AS (
```

```
    SELECT TOP 7 *
```

```
    FROM dbo.AGHealthTrend
```

```
    ORDER BY LogDate ASC
```

```
),
```

```
TrendWithDelta AS (
```

```

SELECT *,
    LogSendMB - LAG(LogSendMB) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY LogDate) AS
DeltaLogSend,
    RedoMB - LAG(RedoMB) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY LogDate) AS DeltaRedo,
    RedoRateMBPerSec - LAG(RedoRateMBPerSec) OVER(PARTITION BY DatabaseName, ReplicaServer ORDER BY
LogDate) AS DeltaRedoRate
FROM RecentTrend
)
SELECT @html += '<h2>AG Lag Trends (Last 7 Snapshots)</h2>' +
    '<table border="1" cellpadding="3" cellspacing="0" style="border-collapse:collapse;">' +
    '<tr style="background-color:#B0C4DE;"><th>Date</th><th>AG</th><th>Replica</th><th>DB</th><th>LogSend
MB</th><th>Trend</th><th>Redo MB</th><th>Trend</th><th>RedoRate
MB</th><th>Trend</th><th>State</th></tr>' +
    STRING_AGG(
        '<tr>' +
        '<td>' + CONVERT(VARCHAR, LogDate, 120) + '</td><td>' + AGName + '</td><td>' + ReplicaServer + '</td><td>' +
DatabaseName + '</td>' +
        '<td>' + CAST(LogSendMB AS VARCHAR) + '</td><td style="background-color:' +
        CASE WHEN DeltaLogSend>10 THEN '#FF6666' WHEN DeltaLogSend>0 THEN '#FFD700' ELSE '#90EE90' END + ';">'
+
        REPLICATE('|', CAST(ISNULL(DeltaLogSend,0) AS INT)) + '</td>' +
        '<td>' + CAST(RedoMB AS VARCHAR) + '</td><td style="background-color:' +
        CASE WHEN DeltaRedo>10 THEN '#FF6666' WHEN DeltaRedo>0 THEN '#FFD700' ELSE '#90EE90' END + ';">' +
        REPLICATE('|', CAST(ISNULL(DeltaRedo,0) AS INT)) + '</td>' +
        '<td>' + CAST(RedoRateMBPerSec AS VARCHAR) + '</td><td style="background-color:' +
        CASE WHEN DeltaRedoRate< -1 THEN '#FF6666' WHEN DeltaRedoRate<0 THEN '#FFD700' ELSE '#90EE90' END +
';">' +
        REPLICATE('|', CAST(ABS(ISNULL(DeltaRedoRate,0)) AS INT)) + '</td>' +
        '<td>' + SyncState + '</td></tr>'
    , '') + '</table>'
FROM TrendWithDelta;

-- =====
-- 5. Send Consolidated HTML Email
-- =====
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'DBMailProfile',
    @recipients = @recipients,
    @subject = @subject,
    @body = @html,
    @body_format = 'HTML';

-- =====
-- 6. Conditional Maintenance (PRIMARY only, no alerts)
-- =====
DECLARE @replica_role NVARCHAR(20);

```

```

SELECT TOP 1 @replica_role = ars.role_desc
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id
WHERE ar.replica_server_name = @@SERVERNAME;

IF @replica_role='PRIMARY' AND @SendMail=0
BEGIN
    PRINT 'PRIMARY replica and no critical alerts. Running maintenance...';
    DECLARE @sql NVARCHAR(MAX) = '';
    SELECT @sql = STRING_AGG(
        'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(t.schema_id)) + '.' + QUOTENAME(t.name) + ' REBUILD
WITH (ONLINE = ON);'
        , CHAR(13)+CHAR(10)
    )
    FROM sys.tables t
    WHERE t.is_ms_shipped = 0;
    EXEC sp_executesql @sql;
    PRINT 'Maintenance completed.';
END
ELSE
    PRINT 'Maintenance skipped due to secondary role or critical alerts.';

```



Key Features

1. **Full AG monitoring:** Health, long-running TX, blocking chains.
2. **Trend tracking with mini bars:** Color-coded trends for **quick visual insight**.
3. **Conditional maintenance:** Runs only if primary and safe.
4. **Single, deployable SQL Agent job:** Automates monitoring, reporting, and maintenance.
5. **Compatible:** SQL Server 2017 → 2025, multiple AG replicas.

This script is **ready to copy-paste into a SQL Agent Job** and schedule daily or more frequently. It gives **real-time and historical insights** while safely maintaining your AG environment.